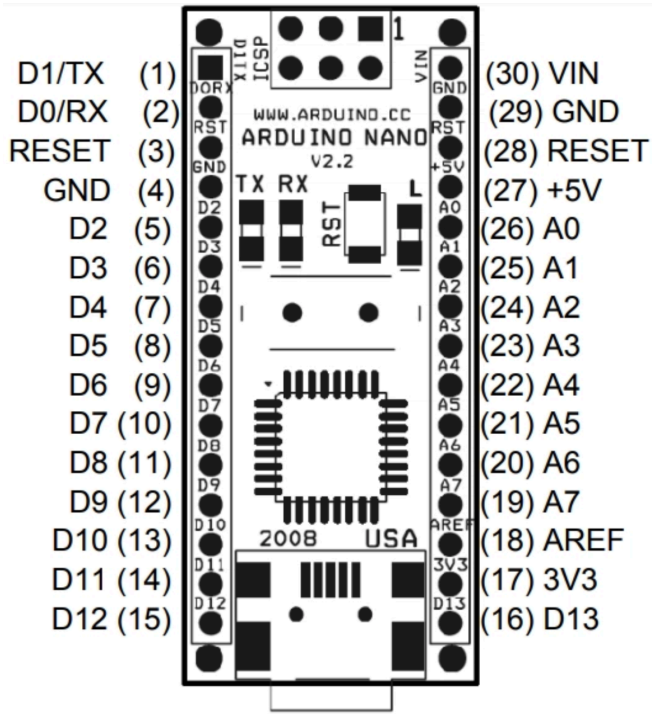
MALMÖ UNIVERSITET

AI Agents in Physical Computing

Through Natural Language Interaction

Anas Aljoudi



Interaction design
One-year Master
15 Credits
VT 2026
Supervisor: Lars Holmberg

Front page image from the Arduino Nano manual.
https://www.arduino.cc/en/uploads/Main/ArduinoNanoManual23.pdf

Abstract
This thesis explores how AI agents and natural language interaction can support beginner interaction with physical computing systems. Using a research through design approach, the project combined literature research, interviews, prototyping, user testing, thematic analysis, and iterative refinement to investigate how users interact with AI assisted Arduino workflows. The research focused on how users shape physical system behavior through natural language, how they interpret AI generated outputs, and how they negotiate control when interacting with physical hardware. The project resulted in a interactive website platform prototype that integrates natural language prompting, AI assisted code generation, wiring visualization, and hardware feedback within a single workflow. The findings show that AI agents reduce barriers related to programming, but users still experience challenges related to wiring, hardware interpretation, and physical understanding. The thesis contributes design knowledge for future AI assisted physical computing systems by highlighting the importance of transparency, visual guidance, and physical understanding during interaction.

Keywords
Interaction Design, Human AI Interaction, Physical Computing, Natural Language Interaction, AI Agents, Command-Line Interfaces, Beginner Electronics.

Acknowledgements
I would like to express my gratitude to my supervisor, Lars Holmberg, for his guidance and support throughout this project. I would also like to thank my brother, Dr. Ahmad Aljoudi, and my classmate, Koen Berkers, for their continuous support and encouragement during the thesis process. I would also like to thank the participants who contributed their time during the interview and testing sessions.

Finally, I would like to acknowledge the use of OpenAI's ChatGPT (GPT-5.5) for grammar and language refinement during the writing of this thesis. AI assisted feedback was used selectively to improve clarity, structure, and readability throughout the writing process. Example prompts included: "*Check the grammar of this paragraph*" and "*Check the spelling of this section*."

Table of Contents

1 Introduction *6*
1.1 Research Questions **6**
1.2 Aim & Contribution **6**
2 Background *8*
2.1 Artificial intelligence and Interaction Design **8**
2.2 Natural Language Interaction **8**
2.3 Prompt Engineering **8**
2.4 AI Agents, Code Generation, and CLI Based Interaction **8**
2.5 Physical Computing and Embodied Interaction **9**
2.6 Human AI Collaboration and Control **9**
2.7 Related Work **10**
3 Methods *14*
4 Limitations *19*
5 Theoretical Framework *20*
6 Design Process *21*
6.1 Literature Research **21**
6.2 Exploring Codex CLI **22**
6.3 Interviews **24**
6.4 Initial Testing **25**
6.5 Thematic Analysis **29**
6.6 Prototyping **30**
7 Design Outcomes Summary *41*
8 Discussion/Evaluation *42*
8.1 AI Agents and Embodied Interaction **42**
8.2 Iterative Interaction, Trust, and Control **42**
8.3 Automation, Transparency, and Learning **43**
8.4 Evaluation of the Prototype and Design Process **44**
8.5 Research Questions Reflection **45**
9 Future work *46*
10 Conclusion *47*
11. References *48*
12. Appendix *50*

1 Introduction

Recent advances in large language models (LLMs) have enabled natural language to function as a primary interface for interacting with computational systems. Within human AI interaction (HAI) and end user programming, this shift reduces technical barriers by allowing users to express intent conversationally rather than through formal code (Lieberman et al., 2006; Myers et al., 2006). More recently, advances in generative AI and large language model based agents have expanded this paradigm by enabling systems not only to generate responses but also to plan, execute code, and interact with external tools through natural language interaction (Shneiderman, 2022; Vaithilingam et al., 2022; Ehsan et al., 2021).

While these developments are well established in software environments, their application to physical computing remains underexplored. Physical computing introduces embodied interaction, where system behavior is expressed through tangible outputs such as light, motion, or environmental change. This creates interaction design challenges related to transparency, causality, feedback, and user control, issues that have been widely discussed within human centered AI and human AI interaction research (Shneiderman, 2022; Ehsan et al., 2021). However, the application of these principles to AI driven physical computing systems remains relatively unexplored within interaction design research.

The integration of AI agents into physical systems requires rethinking interaction design principles related to feedback, transparency, and user agency. When system behavior is both generated by AI and expressed physically, interaction becomes an iterative process of interpretation and adjustment rather than direct manipulation.

Since AI agents operate primarily in digital and computational domains while humans interact through situated and embodied experience, such systems also introduce new forms of human AI collaboration that combine the generative and computational capabilities of AI with human interpretation, contextual understanding, and physical reasoning, reflecting principles of human centered AI (Shneiderman, 2022). This thesis investigates this emerging interaction space by examining how users engage with AI agents that translate natural language into embodied system behavior.

1.1 Research Questions

This thesis is guided by the following research questions:

1. How can users iteratively shape the behavior of a microcontroller based physical system through natural language interaction with an AI agent?
2. How do users interpret and respond to mismatches between intended and AI generated behavior in physical systems?
3. How do users negotiate control with an AI agent when shaping the behavior of a physical system through natural language?

1.2 Aim & Contribution

The aim of this thesis is to explore how interaction design can support user understanding, control, and collaboration when interacting with AI agents that program and control physical systems through natural language input. The project adopts a research through design approach, developing and evaluating a prototype in

which users interact with an AI agent that generates and executes code on a physical device. The contribution of this work lies in producing design knowledge for this emerging interaction paradigm. Specifically, it provides insights into how users interpret AI generated physical behavior, how interaction can support iterative refinement of system outcomes, and how transparency and feedback mechanisms can be designed to improve user understanding and trust.

This thesis is structured as follows. Chapter 2 presents the background and related work relevant to AI agents, natural language interaction, and physical computing. Chapter 3 outlines the methodological approach and research methods used throughout the project. Chapters 4 and 5 discuss the limitations and theoretical framework. Chapter 6 presents the design process, prototype development, testing, and iterative refinement. Chapters 7 and 8 present the final prototype, discussion, and evaluation of the findings. Finally, Chapters 9 and 10 discuss future work and conclude the thesis.

2 Background

This background section outlines the theoretical and technical contexts relevant to the thesis. It introduces key concepts related to artificial intelligence, natural language interaction, AI assisted programming, physical computing, and human AI collaboration. Together, these perspectives establish the foundation for understanding how AI agents can mediate interaction between users and embodied physical systems.

2.1 Artificial intelligence and Interaction Design

Artificial intelligence has shifted interaction design from deterministic systems toward probabilistic and generative behaviors. Traditional interaction design emphasized predictability and direct manipulation, whereas AI driven systems introduce uncertainty and autonomy, requiring designers to address interpretation, trust, and user control. The focus has increasingly moved toward collaboration, where users and systems jointly shape outcomes rather than operate through fixed commands (Amershi et al., 2019). As AI becomes embedded in everyday systems, interaction design must address how users interpret systems that are probabilistic and partially opaque.

2.2 Natural Language Interaction

Natural language interaction enables users to communicate with systems through conversational input, lowering barriers for those without programming expertise. This paradigm is closely related to end user programming, where the goal is to make computational systems accessible to non experts by allowing them to express intent in intuitive ways (Lieberman et al., 2006; Myers et al., 2006). In physical computing contexts, misunderstandings become more visible because natural language input produces physical outcomes. Designing for such interaction therefore requires clear feedback and understandable system behavior (Amershi et al., 2019).

2.3 Prompt Engineering

Prompt engineering refers to crafting textual input to guide the behavior of large language models. In this context, prompts function as a form of indirect programming, where users specify goals and constraints through language rather than formal code (Lieberman et al., 2006). Because small changes in phrasing can produce different outputs, interaction often becomes iterative and exploratory. In systems that generate executable actions, such as Arduino code, users must continuously interpret and refine prompts to achieve desired behavior (Vaithilingam et al., 2022; Shneiderman, 2022).

2.4 AI Agents, Code Generation, and CLI Based Interaction

AI agents extend the capabilities of large language models beyond text generation to include planning, tool use, and code execution. Recent AI assisted programming systems increasingly support natural language driven code generation, allowing users to interact with development environments through conversational input rather than traditional programming syntax (Vaithilingam et al., 2022).

These capabilities are increasingly integrated into command-line interface (CLI) environments. Traditionally, CLI interaction requires technical knowledge and precise syntax, but AI-powered CLI tools such as Claude CLI[1] and similar systems allow users to issue natural language commands that are interpreted and executed by the system. However, this interaction model introduces challenges. The AI agent acts as an intermediary between user intent and execution, which can make system behavior less transparent. Users must interpret how their input is translated into actions, and small changes in phrasing can lead to different results. This is particularly difficult for non programmers, who may lack the technical background required to interpret or debug generated code, a challenge frequently discussed within end user programming research (Lieberman et al., 2006; Myers et al., 2006).

From an interaction design perspective, this highlights the importance of supporting transparency, feedback, and user control. Users need to understand what the system is doing and be able to iteratively refine their input to achieve desired outcomes. These challenges match with broader principles in human AI interaction, emphasizing the need for intelligible and controllable AI systems (Amershi et al., 2019).

2.5 Physical Computing and Embodied Interaction

Physical computing connects digital processes to physical inputs and outputs through sensors and actuators, creating interaction that is embodied and situated (O'Sullivan & Igoe, 2004). The concept of embodied interaction suggests that understanding emerges through physical engagement with systems. Users interpret behavior through perception and action, rather than solely through abstract representations. This perspective has been influential in interaction design research, particularly in understanding how meaning is constructed through experience (Löwgren, 2007). Research within tangible and embodied interaction further argues that physical interfaces support understanding through direct manipulation of material artifacts and spatial interaction, making physical engagement central to interaction design practice (Ishii & Ullmer, 1997).

In AI driven physical systems, natural language input is translated into code and executed as physical behavior. This introduces complexity in how users understand causality and system state. Designing for such systems requires making invisible processes more legible and supporting users in forming accurate mental models (Amershi et al., 2019).

2.6 Human AI Collaboration and Control

As AI systems take on more active roles, interaction shifts toward collaboration between humans and machines, often described as forms of hybrid intelligence where humans and AI systems jointly contribute to problem solving and decision making (Dellermann et al., 2019). Research in HAI highlights the importance of maintaining appropriate levels of user control while leveraging AI capabilities (Amershi et al., 2019). Users must be able to guide system behavior, understand its actions, and intervene when necessary.

[1] Anthropic Claude Code documentation: https://docs.anthropic.com/en/docs/claude-code/overview

At the same time, the system should provide meaningful assistance without requiring detailed technical input. Interaction becomes iterative through continuous prompt refinement and feedback evaluation. For non programmers, this depends heavily on interaction design support and system intelligibility (Zimmerman et al., 2007).

2.7 Related Work

This section presents existing systems and projects relevant to the research area explored in this thesis. The selected examples represent different approaches to AI assisted programming, AI agents, and physical computing, illustrating how natural language interaction is increasingly used to generate, execute, and mediate computational behavior. Together, these examples help position the project within current developments while also highlighting gaps related to embodied interaction, user understanding, and interaction with AI driven physical systems.

2.7.1 GitHub Copilot (AI Assisted Code Generation)

GitHub Copilot[2] is an AI powered coding assistant developed using large language models trained on large scale code datasets (Vaithilingam et al., 2022). It enables users to generate and complete code through natural language prompts or partial input directly within development environments such as Visual Studio Code, as shown in Figure 1. By translating high level intentions into executable code, Copilot represents a significant step toward making programming more accessible and efficient. In software development contexts, generated outputs can often be immediately verified through execution or compilation, making AI generated behavior relatively observable and testable compared to physical computing systems.

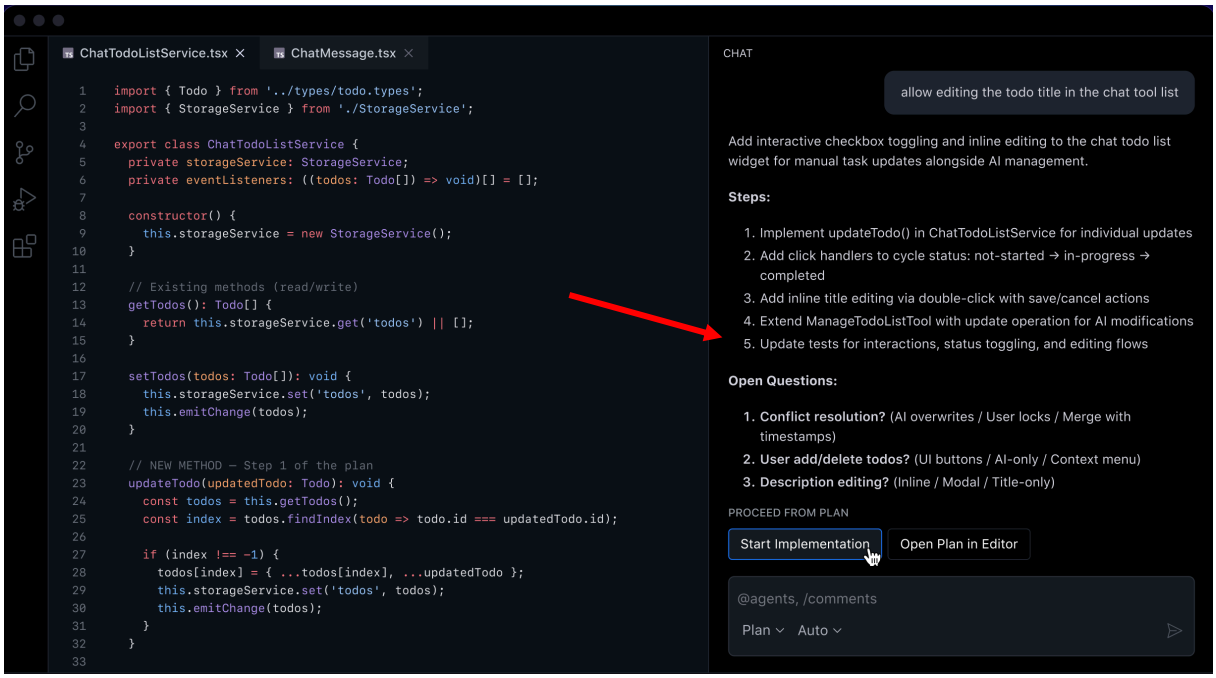


Figure 1: GitHub Copilot generating executable code from a natural language prompt inside Visual Studio Code. The figure highlights how AI generated software outputs can be directly verified through implementation and execution.

[2] GitHub Copilot documentation: https://github.com/features/copilot

2.7.2 ViperGPT: Visual Inference via Python Execution (AI Agents)

ViperGPT is a system that combines large language models with executable code to interpret and act on real world visual input (Surís et al., 2023). Instead of only generating text, the system translates natural language instructions into Python code that interacts with external tools to analyze images and produce results as shown in Figure 2.

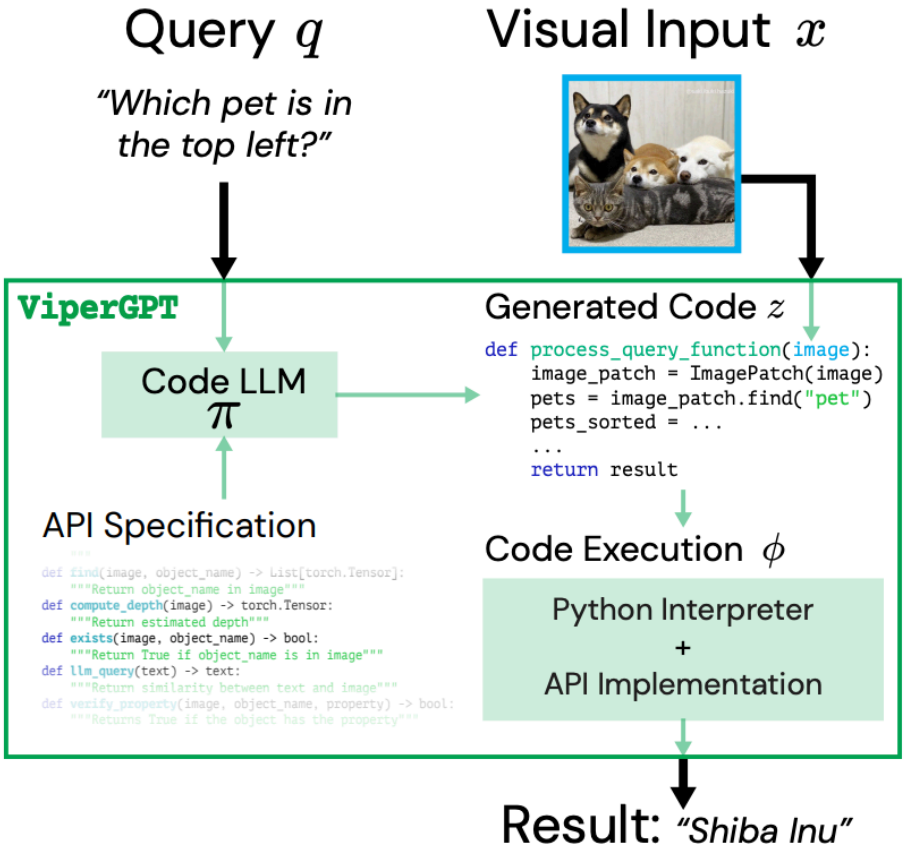


Figure 2: ***ViperGPT system architecture illustrating how natural language queries are translated into executable Python code and processed through external APIs to produce result (*****Surís et al., 2023).**

From an interaction perspective, ViperGPT demonstrates how language can be used to control systems that operate beyond purely textual environments. Users provide natural language instructions, which are interpreted by the model and transformed into executable steps, creating a chain between intent, computation, and observable outcomes as seen in Figure 3.

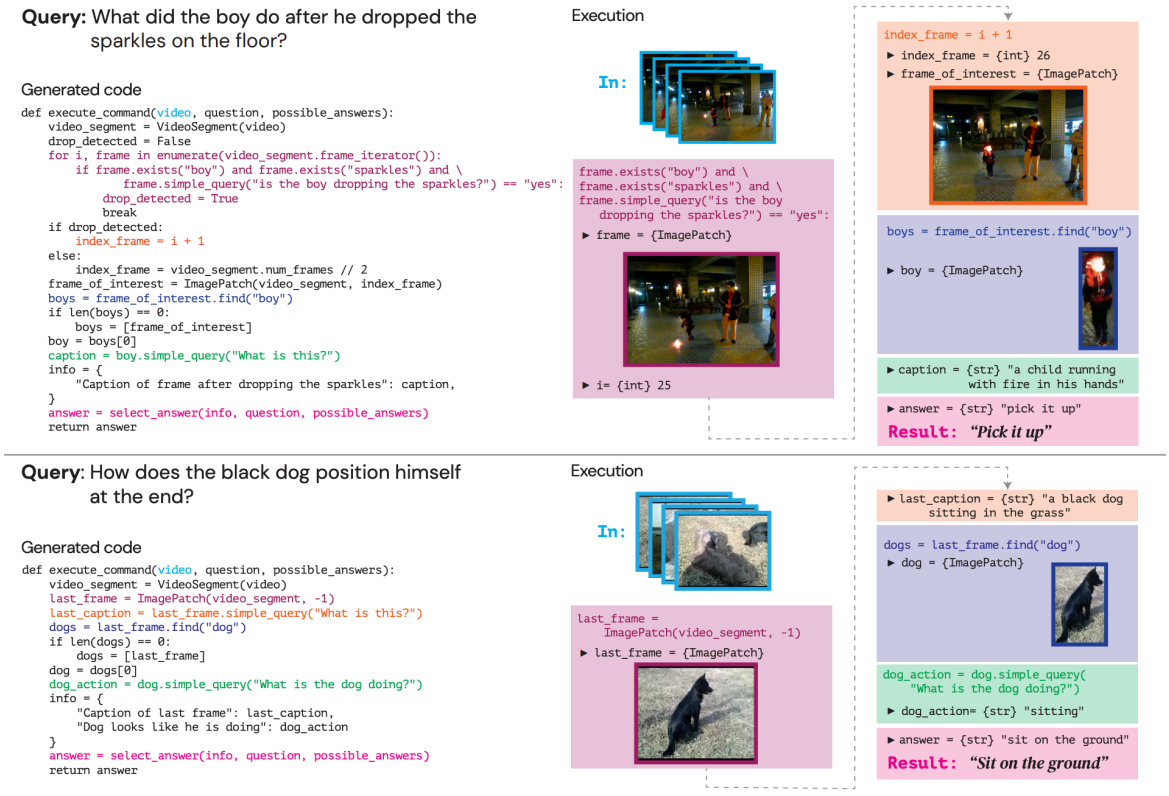


Figure 3: ***Example of ViperGPT execution pipeline, showing generated code, intermediate reasoning steps, and final output derived from visual input (Surís et al., 2023).***

However, while ViperGPT extends AI interaction beyond text, its outputs remain primarily digital, focusing on visual data rather than physical interaction. Users interact with representations of the world (images) rather than directly influencing embodied systems. As a result, the system does not address how users understand or control AI generated behavior when it manifests through physical devices.

The interaction still assumes a level of abstraction in which system processes are not fully transparent to the user. While the generated code can be inspected, understanding how it relates to system behavior requires technical knowledge, limiting accessibility for non programmers. ViperGPT therefore represents an important step toward connecting natural language, AI generated code, and real world interaction. However, it stops short of addressing embodied physical computing, where actions have immediate and tangible effects. This highlights a gap in how AI agents can be designed to support interaction with physical systems in an accessible and intelligible way.

While ViperGPT operates primarily within computer vision contexts rather than physical computing, the system demonstrates how AI agents can coordinate reasoning, code execution, and external tool interaction through natural language instructions. This relates conceptually to the present thesis, where AI agents similarly generate and execute actions within physical computing workflows involving Arduino hardware and user driven interaction.

2.7.3 Arduino Platform (Physical Computing Systems)

The Arduino platform[3] is widely used in physical computing to create interactive systems that connect digital processes with physical inputs and outputs. It allows users to program microcontrollers to control components such as LEDs, sensors, and motors, making it a foundational tool in both education and interaction design practice (Banzi & Shiloh, 2014).

Arduino based systems highlights embodied interaction, where system behavior is directly observable in the physical world. Users experience cause and effect relationships through tangible outputs, which supports learning and experimentation.

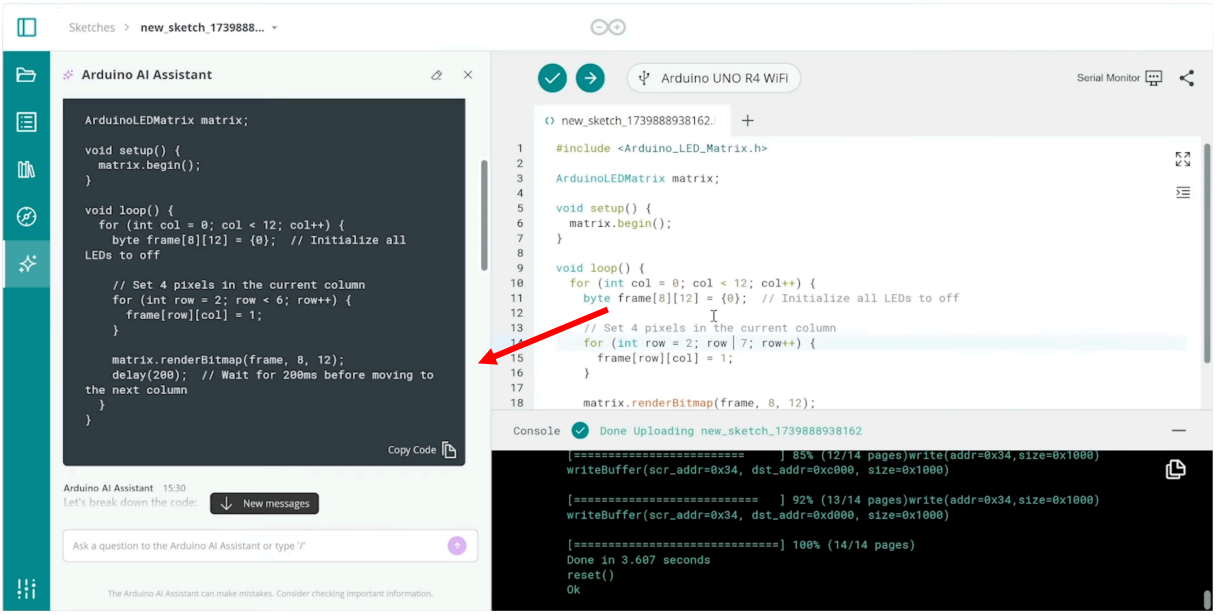


Figure 4: Arduino AI assistant interface generating and uploading code to a microcontroller system[4].

Arduino still requires users to understand programming, hardware configuration, and debugging and physical connections. This creates a barrier for users without programming or electronics experience.

While Arduino demonstrates the potential of physical computing, it lacks integration with natural language interfaces or AI driven behavior generation as shown in Figure 4. Users must manually implement system logic, and there is limited support for higher level interaction where intent can be expressed conversationally and translated into system behavior.

[3] Arduino official website: https://www.arduino.cc
[4] Arduino AI assistant demo: https://www.youtube.com/watch?v=zwwk91yLVxQ

3 Methods

This thesis adopts a qualitative, design oriented methodology grounded in research through design, complemented by literature research, interviews, prototyping and user testing, and thematic analysis. I find that these methods are appropriate for exploring emerging interaction paradigms where knowledge is produced through both design practice and empirical investigation.

3.1 Research through Design

Research through design is a methodological approach in interaction design where knowledge is generated through the iterative process of designing, building, and reflecting on artifacts (Zimmerman et al., 2007). As seen in Figure 5, this approach positions design not only as a means of creating solutions but as a way of producing knowledge about interaction.

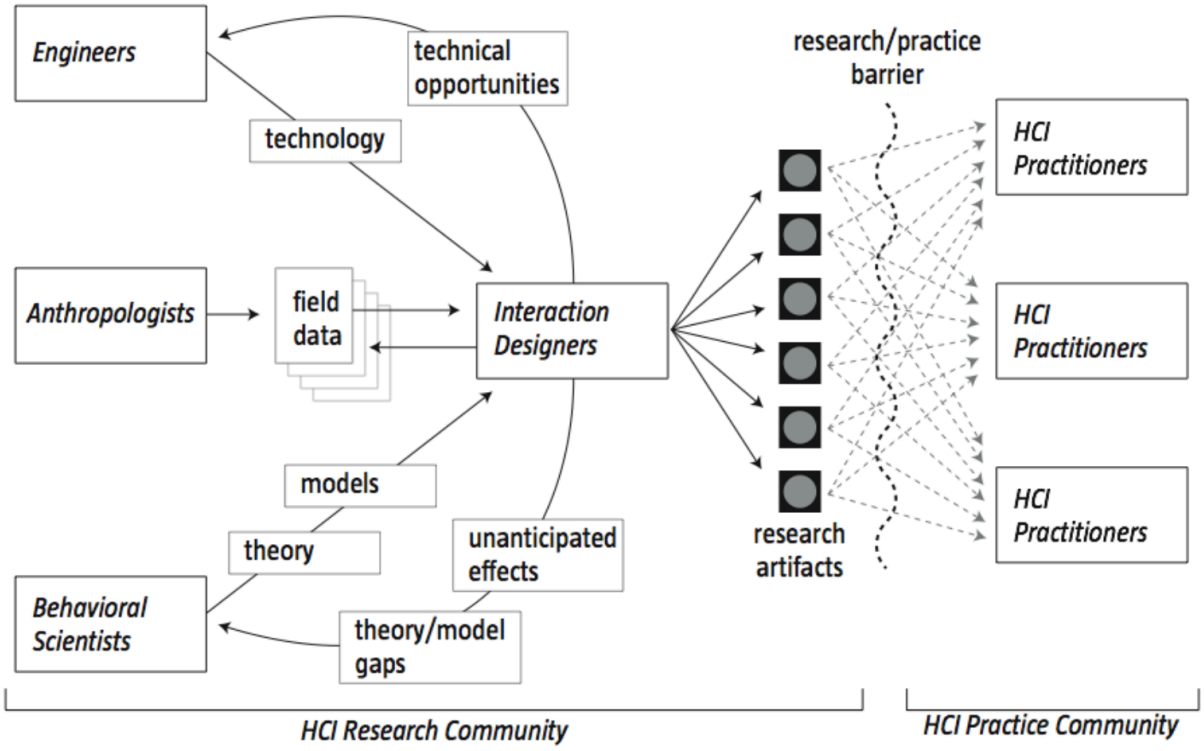


Figure 5: Research through design framework illustrating the relationship between research, design, and artifact production (Zimmerman et al., 2007).

In this thesis, research through design functions as the overarching methodological framework. A series of iterative prototypes are developed to explore how users interact with AI driven physical systems. These prototypes progressively investigate different aspects of interaction, including natural language input, AI generated code execution, feedback, and physical interaction. Insights from each iteration informed later design decisions and prototype refinement.

3.2 Literature Research

Literature research involves the identification, evaluation, and synthesis of academic work to establish a theoretical foundation for a study (Booth et al., 2016). In this thesis, the literature research process was informed by Booth et al. (2016), particularly in relation to identifying and comparing research related to natural language interaction, human AI interaction, and physical computing.

The literature research was conducted through Google Scholar, ACM Digital Library, and IEEE Xplore using keyword combinations related to AI assisted interaction, natural language interfaces, physical computing, research through design, and end user programming. Relevant sources were identified through iterative searching and snowballing techniques by following references between related publications.

3.3 Interviews

Semi structured interviews were conducted to explore participants experiences with programming, physical computing, and AI assisted tools (Kvale & Brinkmann, 2009). Semi structured interviews were conducted with participants who had limited experience with programming and physical computing. The interviews explored experiences with Arduino systems, electronics, programming, and AI assisted tools.

The interviews were documented through written notes taken during each session. Key observations and participant reflections were later reviewed through thematic analysis to identify recurring patterns related to programming barriers, trust in AI generated outputs, and understanding of physical computing concepts.

3.4 Prototyping

Prototyping is a central interaction design method used to explore and evaluate ideas through the creation of artifacts (Houde & Hill, 1997). Prototypes support the exploration of interaction possibilities and iterative refinement of system behavior. As shown in Figure 6, prototyping can be understood through dimensions such as role, look and feel, and implementation, which shape how a prototype communicates design ideas.

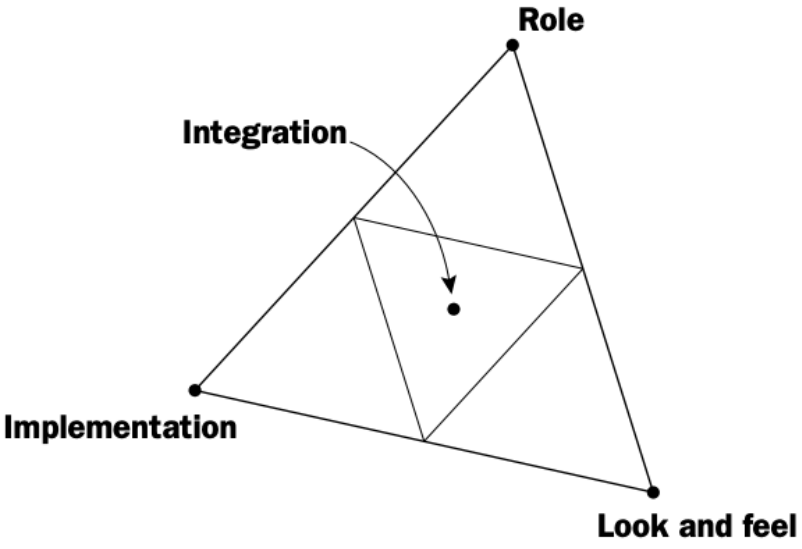


Figure 6: Dimensions of prototyping (Houde & Hill, 1997).

In this thesis, a functional prototype is developed that enables users to control a physical system through natural language interaction with an AI agent. The prototype serves as a research instrument, allowing the exploration of how AI generated code translates into physical behavior and how interaction design can support user understanding and control.

3.5 User Testing & Think Aloud Method

User testing and think aloud sessions were conducted to examine how participants interacted with the prototype and interpreted AI generated behavior (Ericsson & Simon, 1993; Wenger & Spyridakis, 1989). The testing sessions were documented through written notes taken during interaction. Participant observations, verbal reflections, and recurring behaviors were later reviewed and organized thematically to identify usability challenges, interaction patterns, and responses to AI generated system behavior.

In this thesis, user testing sessions are conducted with participants who have little or no programming experience. During these sessions, participants are asked to think aloud while interacting with the prototype, expressing their expectations, reasoning, and reactions in real time. This approach enables the identification of how users interpret system behavior, how they respond to mismatches between intended and actual outcomes, and how they iteratively refine their interaction with the AI agent.

3.6 Thematic Analysis

Thematic analysis is a qualitative method used to identify and interpret recurring patterns within data (Braun & Clarke, 2006). In this thesis, thematic analysis was applied to interviews, think aloud sessions, and prototype evaluations. Participant reflections, observations, and interaction behaviors were reviewed and coded to identify themes related to programming barriers, trust, interpretation, and embodied interaction within AI assisted physical computing.

3.7 Ethical Considerations

This thesis project was conducted in accordance with established ethical research guidelines and regulations. The study follows the ethical principles outlined by the Swedish Research Council (2017) and complies with the General Data Protection Regulation (European Union, 2016).

Participants involved in interviews and user testing were fully informed about the purpose of the study, the nature of their participation, and their rights before taking part. All five participants signed a consent form prior to participation, which is included in the appendix of this thesis (see Appendix A). Participation was voluntary, and participants had the right to withdraw at any time without consequence.

No sensitive personal data was collected during the study. All data was anonymized and handled in a secure manner to ensure participant privacy and confidentiality. In accordance with GDPR (European Union, 2016), only authorized individuals had access to the collected data, and it was used solely for the purposes of this research. The study adhered to principles of transparency, responsibility, and respect for participants throughout the research process, ensuring that all interactions were conducted in a safe and ethically sound manner.

3.8 Target Group

The target group for this project consists of individuals without formal programming or electronics expertise who are nevertheless familiar with interacting with digital systems. This includes students and users from non technical backgrounds who may have prior exposure to AI based tools such as chatbots but lack the ability to confidently write or understand code. The focus is on users who can articulate intentions through natural language but are not trained in programming or electronics.

To investigate beginner interaction behaviors within AI assisted physical computing, the study involved five participants from Interaction Design related educational[5] backgrounds with varying levels of familiarity with Arduino programming and physical computing, as shown in Table 1. Although all five participants had prior exposure to digital design and interactive systems through their education, none had advanced programming or electronics expertise. This allowed the project to investigate how users with creative and design oriented backgrounds interact with AI assisted physical computing systems while still representing beginner level technical experience. Participants generally relied on exploratory and iterative interaction strategies, often expressing goals through informal natural language rather than technical terminology. Participants are referenced throughout the findings using anonymized identifiers (P1–P5) to connect observations and quotations to individual participants while maintaining confidentiality.

The same five participants were intentionally involved throughout the interviews, initial testing sessions, and later prototype evaluation stages. This decision was made to support continuity across the iterative research through design process and to allow changes in participant interaction, understanding, and behavior to be observed over time. The participants were recruited through convenience sampling and were not personally known to the researcher prior to the study. Using the same participant group across multiple stages may have influenced the findings through increased familiarity with the prototype and interaction process, which is further reflected upon in the discussion section.

Table 1: Participant demographics and experience with physical computing

Participant	Age	Gender	Background	Experience
P1	27	Female	Interaction Design Student	Beginner
P2	26	Male	Interaction Design Student	Limited Experience
P3	23	Female	Interaction Design Student	Beginner
P4	25	Female	Interaction Design Student	Beginner
P5	23	Male	Interaction Design Student	Limited Experience

[5] Malmö University, *Interaction Design Bachelor Programme* syllabus: https://utbildningsinfo.mau.se/program/utbildningsplan/en/f6c0ed35-78f3-11f0-b4fd-2140da76402d/20262

An excluded target group consisted of individuals with advanced programming or electronics expertise. Such users were excluded, as their prior knowledge could significantly influence how they interpret system behavior and reduce the relevance of the findings for beginner oriented interaction. Demographic factors such as age, gender, or occupation were not used as primary selection criteria. Instead, the study prioritized participants' level of technical experience and interaction behavior, ensuring that insights are grounded in how non experts engage with AI driven physical systems.

3.9 Usage of AI

Artificial intelligence plays a central role in both the development and evaluation of the prototype in this thesis. The project utilises AI models capable of translating natural language into executable code, enabling users to interact with a physical system without requiring programming knowledge.

For implementation and testing, Codex CLI[6] is used as the primary tool. Codex CLI is an AI powered coding agent operating within a command line interface, allowing users to generate, modify, and execute code through natural language input. Within this project, it functions as an intermediary between user intent and system execution, translating user prompts into code that is run on the physical system.

The system is built using GPT-3.5 Turbo model accessed through the OpenAI API, which are used to interpret user input and generate corresponding code. These models are applied iteratively throughout the design process to test different prompting strategies, evaluate system responses, and refine how user intentions are translated into physical behavior.

The use of AI in this thesis is both technical and investigational. It enables the implementation of the prototype while also allowing the study to examine how users interact with AI driven systems that exhibit generative and partially autonomous behavior. In this context, AI is treated as both a design material and a subject of inquiry, supporting the exploration of interaction design challenges related to understanding, control, and collaboration in physical computing systems.

[6] OpenAI Codex CLI documentation: https://developers.openai.com/codex/cli

4 Limitations

This study is exploratory and grounded in a research through design approach, meaning that the focus is on generating situated design knowledge through iterative prototyping and testing rather than producing universally generalisable results. The project investigates an emerging interaction space between AI agents, natural language interaction, and physical computing, where both the technology and interaction paradigms are still rapidly evolving.

Another limitation relates to the use of AI models and AI assisted development tools. Systems such as Codex CLI and GPT based models are probabilistic and continuously evolving, meaning that outputs may vary across interactions and over time. Consequently, the findings reflect interaction with a specific configuration of tools during the period of the study rather than a fixed technological environment.

The project is also dependent on external AI infrastructure and token based usage limitations. Since Codex CLI and related AI systems rely on API access and usage quotas, access to certain models may become limited during development and testing. In addition, the final prototype may require integration through external API keys, introducing technical and economic constraints related to model availability and operational cost. Due to these constraints, the implemented prototype may rely on smaller and more cost efficient models rather than the latest high performance models used during experimentation.

Finally, the prototype evolved iteratively throughout the project, meaning that different stages of testing explored different levels of technical and physical complexity. Early testing focused on interaction with the AI system itself, while later prototype iterations expanded toward more complex physical computing scenarios involving external components and wiring.

5 Theoretical Framework

This thesis is situated within interaction design research at the intersection of human AI interaction, natural language interfaces, and embodied interaction. The theoretical framework provides conceptual perspectives for understanding how users interpret, negotiate, and interact with AI generated behavior in physical computing contexts.

A central perspective is the shift from direct manipulation toward interaction through intent. Within end user programming and human AI interaction research, natural language interfaces reduce technical barriers by allowing users to express goals conversationally rather than through formal programming syntax (Lieberman et al., 2006; Myers et al., 2006). In AI assisted systems, this changes the role of the user from manually constructing behavior to iteratively refining outcomes generated by AI systems (Vaithilingam et al., 2022).

The thesis also draws on theories of embodied and tangible interaction, where understanding emerges through physical engagement with systems and environments (Dourish, 2001; Löwgren, 2007; Ishii & Ullmer, 1997). In physical computing, interaction is experienced through tangible outputs such as light, sensors, and movement, meaning that users interpret system behavior not only cognitively but through situated and material interaction.

These perspectives are further connected to research on human AI collaboration and control, where interaction is understood as a continuous negotiation between user intention and system autonomy (Amershi et al., 2019; Shneiderman, 2022). Rather than functioning as passive tools, AI agents actively participate in generating actions and outcomes, requiring users to interpret, evaluate, and refine system behavior over time.

Together, these theoretical perspectives provide a framework for analyzing how users understand AI generated behavior, how they interact with AI agents through natural language, and how meaning and control are negotiated when interaction extends into embodied physical systems.

6 Design Process

This section presents the design process and findings in a sequential structure following the iterative stages of the research through design process. Before the development of the first prototype, an initial literature research and technical exploration of Codex CLI were conducted (Section 6.1 and Section 6.2) to establish a theoretical and technical understanding of AI driven interaction in physical computing contexts. These early investigations informed the subsequent interviews and initial testing sessions (Section 6.3 and Section 6.4), which explored how non programmers perceive, interpret, and interact with AI-assisted systems.

The findings from these stages were then synthesized through thematic analysis (Section 6.5), forming the basis for later design decisions and prototype development. The same five participants (P1–P5) were involved throughout the interviews, initial testing sessions, and later prototype evaluation stages. This allowed participant behavior, interaction patterns, and reflections to be compared across multiple stages of the research through design process while maintaining consistency and anonymization throughout the analysis.

6.1 Literature Research

The design process began with a literature research to identify central themes, challenges, and opportunities within human AI interaction, natural language interfaces, and physical computing. The literature research aimed to establish both a theoretical and practical understanding of how users interact with systems that translate natural language intent into computational and physical actions.

The initial literature research identified recurring themes related to natural language interaction, end user programming, explainable AI, and embodied interaction within physical computing contexts. Previous research highlighted how conversational AI systems can reduce technical barriers for non programmers while also introducing challenges related to transparency, trust, and interpretation of system behavior (Lieberman et al., 2006; Shneiderman, 2022; Ehsan et al., 2021). Research within physical computing further emphasized the importance of embodied interaction and the difficulties beginners experience when translating abstract instructions into physical and spatial actions (Dourish, 2001; O'Sullivan & Igoe, 2004).

A central finding identified through the literature research was the shift from direct manipulation toward intent based interaction, where users no longer construct behavior step by step, but instead describe goals that are interpreted and executed by AI systems. Research on end user programming and AI assisted interaction suggests that conversational interaction can lower technical barriers and enable non programmers to engage with computational systems through natural language input rather than formal syntax (Lieberman et al., 2006; Shneiderman, 2022).

At the same time, the literature highlights that such systems introduce new interaction challenges related to transparency, predictability, feedback, and user control. These challenges connect back to the background discussion on human AI interaction and collaborative control, where AI systems are understood as active participants in interaction rather than passive tools.

Research on embodied interaction and physical computing highlighted how users interpret system behavior through tangible outputs such as light, movement, and sensor responses (Dourish, 2001; O'Sullivan & Igoe, 2004). The literature also showed that limited research has explored how users understand AI generated behavior in physical computing contexts.

Together, these findings revealed a central tension that informed the direction of the project: while natural language interaction makes programming more accessible, interaction with physical systems still depends on understanding hardware behavior, system causality, and physical configuration. This suggested that the primary challenge may not lie in generating code itself, but in helping users interpret and manage the relationship between abstract instructions and embodied execution.

These insights informed the subsequent stages of the design process, including the exploration of Codex CLI, the interviews, the initial testing sessions, and the later prototype development and evaluation stages.

6.2 Exploring Codex CLI

The exploration of Codex CLI functioned as both a technical investigation and a personal process of learning how AI agents operate within command line environments. Rather than approaching the system solely as a development tool, the exploration focused on understanding what kinds of interaction behaviors, assumptions, and challenges emerge when natural language is used to generate and execute code for physical systems.

The exploration of Codex CLI was also motivated by the rapid recent development of large language models capable of supporting programming workflows through natural language interaction. Although earlier AI coding systems existed, recent advances in generative AI models during 2024 significantly improved their ability to generate, interpret, and iteratively refine executable and verifiable code[7] across practical programming domains. This shift contributed to the growing relevance of AI assisted development tools such as Codex CLI within both professional and educational programming contexts.

Early experimentation involved learning how prompts influence system behavior, how the agent interprets instructions, and how actions are executed within the terminal environment. This included testing different prompting strategies, slash commands, plugins, and command based workflows to understand the boundaries and possibilities of the system. Repeated interaction with the system indicated that effective use required more than simply writing requests in natural language. Small changes in phrasing often produced different results, requiring iterative refinement and continuous interpretation of system feedback.

The exploration revealed a gap between the perceived simplicity of natural language interaction and the actual cognitive and technical demands placed on the user. While the system initially appears accessible due to its conversational interaction style,

[7] Andrej Karpathy discusses how recent GPT-5.4 class models made AI significantly more effective within "verifiable domains" such as programming, where outputs can be tested through execution. See *Andrej Karpathy: From Vibe Coding to Agentic Engineering*: https://youtu.be/96jN2OCOfLs?t=44

successful use still depends on understanding system behavior, execution flows, and terminal based interaction patterns. Interacting with the AI agent therefore became less about “asking for code” and more about maintaining an ongoing dialogue with the system. This interaction pattern connects back to the discussion of AI agents as intermediaries between user intention and system execution, where interaction becomes iterative and negotiation based rather than directly controlled.

The exploration also revealed how the command line interface itself shapes perception and usability. Initially, the interface appeared dense, technical, and difficult to navigate, even during personal experimentation. Over time, however, familiarity with the interaction flow reduced this complexity, shifting attention from the interface itself toward the behavior of the AI agent. This highlights how usability in AI driven CLI environments is closely tied to experience, interpretation, and successful interaction outcomes rather than visual simplicity alone.

These observations became important in shaping the direction of the project. They suggested that the primary challenge for non technical users may not be generating code itself, but understanding how AI generated actions relate to system behavior and physical execution. To further explore these interaction challenges, a set of key technical concepts required for interacting with AI driven physical computing systems were identified and evaluated based on my own assessment of accessibility for non technical users, as shown in Table 2.

Table 2: Required Technical Concepts for Interaction

Concept	Description	Relevance to System	Difficulty for Non Technical Users
Programming	Understanding logic, structure, and flow of code	Needed to interpret or debug AI generated code	High
Command Line (CLI)	Interaction through terminal based commands	Core interface for using Codex CLI	High
Prompting	Writing natural language instructions for the AI	Primary interaction method	Low–Medium
System Feedback	Interpreting outputs, logs, and system responses	Refining interaction	Medium
Components	Physical elements such as LEDs, sensors, and actuators	Required for building the physical system	Medium
Wiring	Connecting hardware components through circuits	Necessary for enabling physical interaction	Medium
Voltage	Electrical power requirements of components	Important for correct and safe operation	Medium

6.3 Interviews

Semi-structured interviews were conducted with five participants. All five participants were asked the same set of seven questions, focusing on their experience with programming, physical computing, and expectations of AI assisted tools. The full interview questions is provided in Appendix B. The interviews aimed to establish a baseline understanding of participants' perceived competence, mental models, and interaction challenges prior to engaging with the system.

Across all five participants, a consistent pattern of low perceived competence emerged. Three participants rated themselves at the lowest level of experience, while two reported slightly higher confidence in programming only. Despite having completed multiple assignments, participants did not demonstrate a stable sense of understanding. Instead, their competence appeared situational and dependent on external support.

Recurring patterns across the interviews revealed a distinction between programming and wiring. Although programming was initially identified in Table 2 as highly difficult for non technical users, participants often described it as more manageable in practice due to the availability of AI tools. All five participants reported being able to modify and experiment with generated code even without fully understanding it. P5 described this process as relying on AI to "*generate code and then adjusting it until it works*," (P5) highlighting a trial and error approach supported by external tools. This suggests that AI assisted interaction may reduce some of the barriers traditionally associated with programming by shifting interaction from manual code construction toward intention based prompting and experimentation.

In contrast, wiring was consistently identified as the most challenging aspect, despite being initially evaluated in Table 2 as only medium difficulty for non technical users. The interviews therefore revealed a mismatch between the earlier exploratory evaluation and participants' actual experiences during interaction.

All five participants described wiring as unclear, rigid, and difficult to troubleshoot. P4 stated that wiring errors were "*harder to figure out because nothing tells you what went wrong*" (P4), while P1 described the process as something they could follow but not fully understand: "*I can connect it if I see an example, but I don't know why it works"* (P1). This reflects the embodied interaction perspective discussed earlier, where users must connect abstract representations and instructions to physical and spatial behavior.

A recurring issue across all interviews was the absence of a coherent mental model of system behavior. When asked what happens during code upload, two participants consistently described the process as unclear or automatic, often referring to it as "*magic.*" This also corresponds with the earlier discussion on transparency and intelligibility in AI driven systems, where system behavior can become difficult for users to interpret or explain. Even participants with limited prior programming experience (P2 and P5) were unable to explain how code translates into physical output.

All five participants also demonstrated a strong dependency on external support systems. AI tools were primarily used for programming tasks, while wiring relied on examples, peers, or instructors. Trust in AI was therefore domain specific. While two participants expressed confidence in AI generated code, they were hesitant to rely on AI for physical setup. P5 noted: “I trust it for code, but not for wiring because I can’t see if it’s correct” (P5).

Expectations of AI tools revealed an important limitation. All five participants anticipated that tools like Codex CLI would assist with programming, but not necessarily with physical interaction. Four out of five participants highlighted the need for visual or step by step guidance, suggesting that text based interaction alone is insufficient for tasks involving spatial and embodied understanding.

The interview material was later reviewed through thematic analysis, where recurring patterns, participant concerns, and interaction perspectives were identified across the collected notes and observations. Themes related to programming uncertainty, wiring interpretation, trust in AI generated outputs, and reliance on iterative experimentation informed the later testing and prototype development stages.

6.4 Initial Testing

Based on the literature research, technical exploration of Codex CLI, and the interview findings, an initial testing phase was conducted to investigate how users interact with AI agents in physical computing contexts. Rather than using a custom developed prototype, participants interacted directly with Codex CLI through a command line interface connected to an Arduino Nano 33 BLE Sense[8]. Before testing began, Codex CLI was configured through an initial system style prompt written at the beginning of the interaction session to constrain the AI toward Arduino specific behavior and hardware interaction. Participants then used natural language prompts to generate and execute Arduino code, allowing the study to observe how users interpreted AI generated instructions, system feedback, and physical hardware behavior during interaction. The initial Codex CLI setup used during testing is shown in Figure 7.

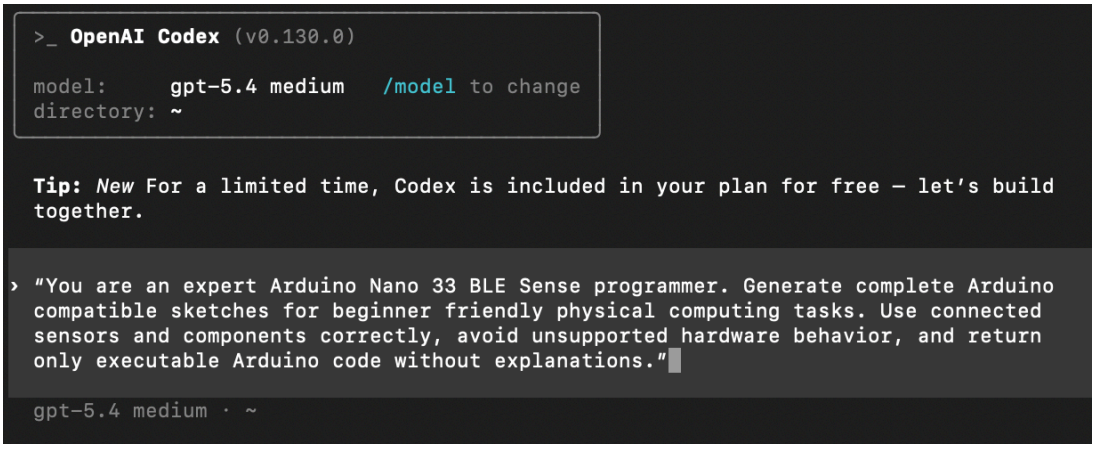


Figure 7: Initial Codex CLI setup used during initial testing, including the Arduino focused configuration prompt used to guide AI generated behavior during interaction.

[8] Arduino Nano 33 BLE Sense documentation: https://docs.arduino.cc/hardware/nano-33-ble-sense/

The initial testing setup was intentionally limited to the built in features of the Arduino board, such as the onboard LED and sensors, without requiring additional external components or wiring. This decision was made to reduce physical complexity during the first testing phase and instead focus on how participants interacted with the AI agent and understood the relationship between prompts, generated code, and physical output.

Following the interviews, participants engaged in initial testing using Codex CLI. Each participant was asked to complete a simple task, with different tasks assigned to capture a range of interaction scenarios (see Appendix C). The tasks were performed using the Arduino Nano 33 BLE Sense and relied only on the board's built in functionality, including the onboard LED and temperature sensor. During the sessions, participants were also asked to think aloud while performing the tasks, verbalizing their thoughts, expectations, and reactions in real time.

The testing sessions generated observational and reflective data that was later analyzed through thematic analysis to identify recurring interaction patterns, participant responses to system feedback, and decision making behaviors during interaction. The interaction process connects back to the earlier discussion on iterative interaction, where users continuously refine prompts and actions based on system feedback. All five participants frequently experienced information overload when interacting with the CLI environment and focused mainly on actionable prompts rather than interpreting system feedback.

Rather than processing this information fully, four out of five participants adopted selective attention strategies, focusing mainly on actionable elements such as confirmation prompts or explicit instructions. This behavior reflects the challenges of transparency and intelligibility previously discussed in relation to AI mediated interaction. As P2 stated: "*there's a lot happening here, I just look for what I need to click or confirm*" (P2).

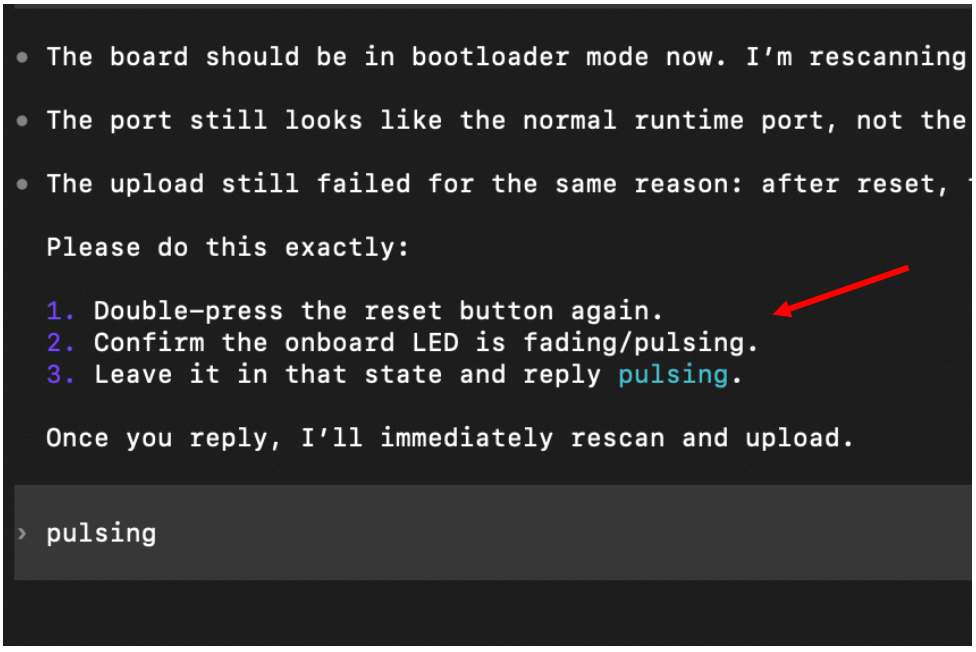


Figure 8: CodeX CLI interaction showing continuous system feedback during task execution. Participants primarily focused on actionable prompts such as "ready" and "pulsing."

Four out of five participants interacted reactively, often approving actions without fully understanding system behavior. Most focused on achieving functional outcomes rather than interpreting the underlying processes. P2 explained: *"I just keep pressing yes so it continues"* (P2). highlighting how progress becomes decoupled from comprehension. Rather than developing a deeper understanding of system behavior, participants focused primarily on achieving functional outcomes, reinforcing the interaction patterns described in the theoretical framework.

Despite these challenges, all five participants were able to complete the assigned tasks. This reveals a critical contradiction: the system is functionally effective but experientially demanding. Once participants reached a successful outcome, their perception of the system shifted significantly. What was initially described as confusing or overwhelming became perceived as efficient and useful. As P3 reflected: *"now that it worked, it actually feels easier than switching between tools"* (P3).

Four out of five participants highlighted advantages compared to their previous workflows, particularly regarding the ability to remain within a single interaction environment instead of switching between tools such as ChatGPT and the Arduino IDE during programming tasks. Four participants described the continuous interaction within a single interface as more efficient, especially when iterating through natural language prompts. However, this positive perception depended on reaching a working result. All five participants indicated that without guidance, they would likely have abandoned the process earlier. P4 noted, "*if I didn't get help at the start, I would probably stop*" (P4), pointing to the fragility of initial interaction.

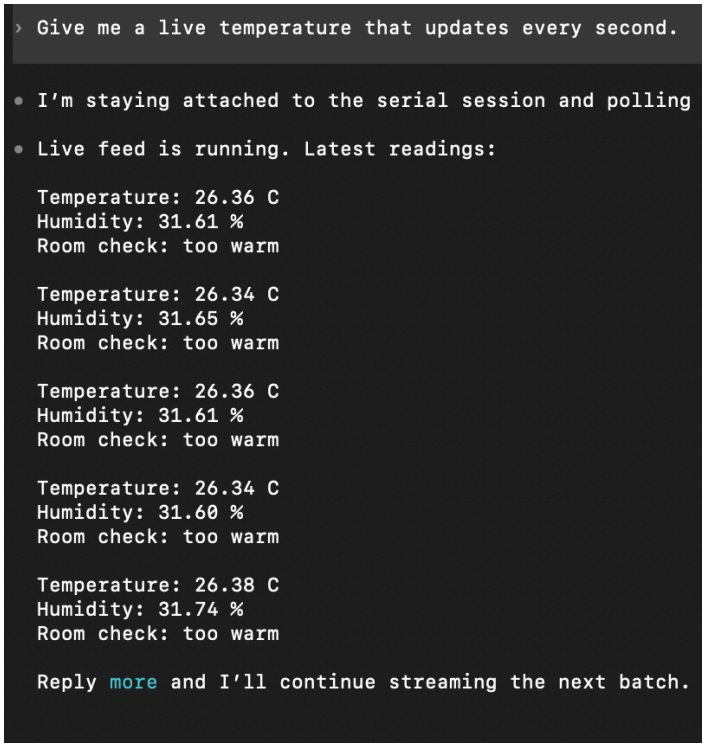


Figure 9: Codex CLI output showing successful task completion through continuous sensor data streaming.

Themes related to prompting behavior emerged as central to successful interaction. All fine participants initially produced vague or incomplete prompts, leading to suboptimal results. Over time, they refined their input, becoming more specific and structured. This was reflected in comments such as P5 explained: "*I realized I need to be more clear with what I want*" (P5). demonstrating an evolving understanding of how to communicate with the system.



Figure 10: A participant prompting Codex CLI to generate a wiring diagram for connecting an Arduino Nano 33 BLE to an LED ring. The system responds with a text-based representation of the circuit and connection layout.

Wiring remained a recurring source of uncertainty, although participant reflections suggested a gradual shift in perception during interaction. While participants initially distrusted AI support for physical tasks, interaction with the system created tentative openness. The ability to receive step by step instructions led some participants to reconsider the role of AI in supporting physical interaction. P1 expressed this shift by stating. P1 suggested: *"maybe it could help if it shows it step by step"* (P1) indicating emerging trust under certain conditions.

Four out of five participant perceptions of the interface also shifted throughout interaction. The command line interface was initially described as intimidating and difficult to navigate. However, after successful use, participants began to appreciate its directness and efficiency. As P3 reflected: "*it looks complicated at first, but once you know what to do, it's actually straightforward"* (P3). This suggests that perceived usability is not fixed, but emerges through successful interaction. Similar to the interview material, observations and participant reflections collected during the think aloud testing sessions were reviewed through thematic analysis. Recurring interaction patterns related to interface confusion, prompting behavior, technical uncertainty, and reliance on confirmation based interaction informed the subsequent prototype iterations and interface decisions.

6.5 Thematic Analysis

Thematic analysis was applied across both the interview material and the think aloud testing sessions to identify recurring interaction patterns, participant concerns, and behavioral tendencies throughout the design process.

The thematic analysis reveals a set of interconnected patterns as shown in Figure 11 that describe how participants interact with AI assisted physical computing. Rather than isolated issues, the findings indicate a consistent misalignment between understanding, interaction behavior, and the translation of abstract instructions into physical execution.

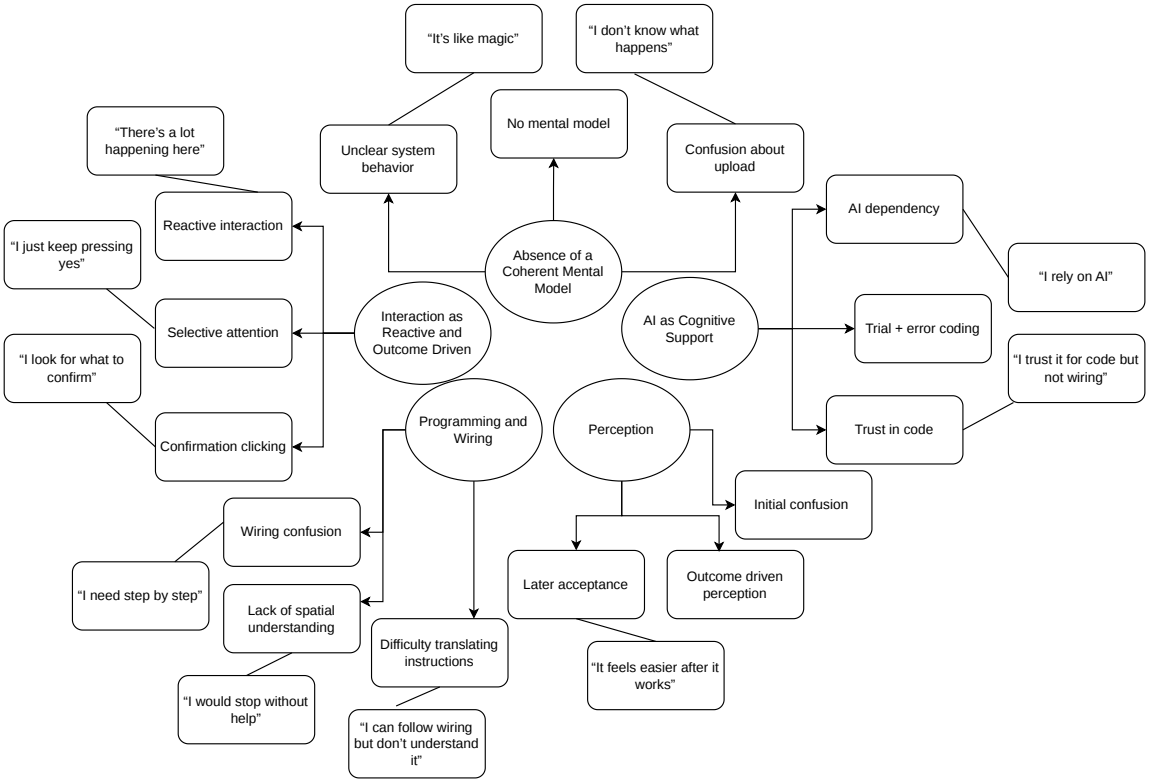


Figure 11: ***Thematic map illustrating data from interviews and testing. The diagram highlights how user interaction with AI driven physical systems emerges through interconnected patterns (Braun & Clarke, 2006).***

A central theme emerging from the interviews and testing sessions was participants' limited or fragmented understanding of system behavior. Participants often struggled to explain how code execution translated into physical outcomes, suggesting the absence of a coherent mental model of how the system operated.

Interaction therefore appeared to be primarily procedural rather than conceptual, with users completing tasks without fully understanding the underlying processes involved.This connects back to the earlier discussion on AI driven systems and human AI collaboration, where interaction increasingly shifts toward outcome oriented negotiation rather than direct understanding and control.

This lack of understanding relates directly to interaction as reactive and outcome driven. Four out of five participants focus on progressing through the system rather

than interpreting it, engaging in confirmation based actions and selectively attending to information that enables task completion. The interface is experienced as dense, leading users to filter information rather than fully engage with it.

This reflects the previously discussed challenges of AI mediated interaction, where system logic and execution processes become difficult for non technical users to interpret. A third theme highlights AI as cognitive support rather than a tool for understanding. Participants rely heavily on AI to generate and modify code, often through iterative trial and error. While this enables task completion, it does not contribute to deeper comprehension, reinforcing dependency on external tools.

This dynamic becomes more pronounced in the asymmetry between programming and wiring. Programming is perceived as flexible and supported by AI, while wiring is experienced as rigid and difficult. Participants struggle with spatial reasoning and translating instructions into physical configurations, revealing a disconnect between abstract representations and embodied interaction. This directly reflects the embodied interaction perspective discussed throughout the background and theoretical framework, where meaning and understanding emerge through physical engagement with systems. Lastly, perception is outcome driven, where initial uncertainty shifts toward acceptance once a working result is achieved. Usability is therefore shaped by successful outcomes rather than clarity during interaction.

Across the themes, participants were generally able to generate and modify code through AI assistance but continued to struggle with wiring and physical interaction. The findings suggest that the primary challenge in beginner oriented physical computing is not code generation itself, but supporting understanding of physical system behavior and hardware configuration.

6.6 Prototyping

In response to the findings from the interviews, initial testing, and thematic analysis, the prototype was developed as a website interactive platform designed to support beginners in creating Arduino projects through natural language interaction, AI assisted code generation, and visual hardware guidance. The system combines software interaction and physical computing within the browser, allowing users to progress through the development process step by step.

The prototype was designed to reduce the complexity associated with Arduino programming and wiring by structuring interaction into sequential stages that guide users from idea generation to physical execution.

This design direction emerged directly from the interviews, initial testing sessions, and thematic analysis, where participants consistently described uncertainty regarding system behavior, wiring, and interaction flow.

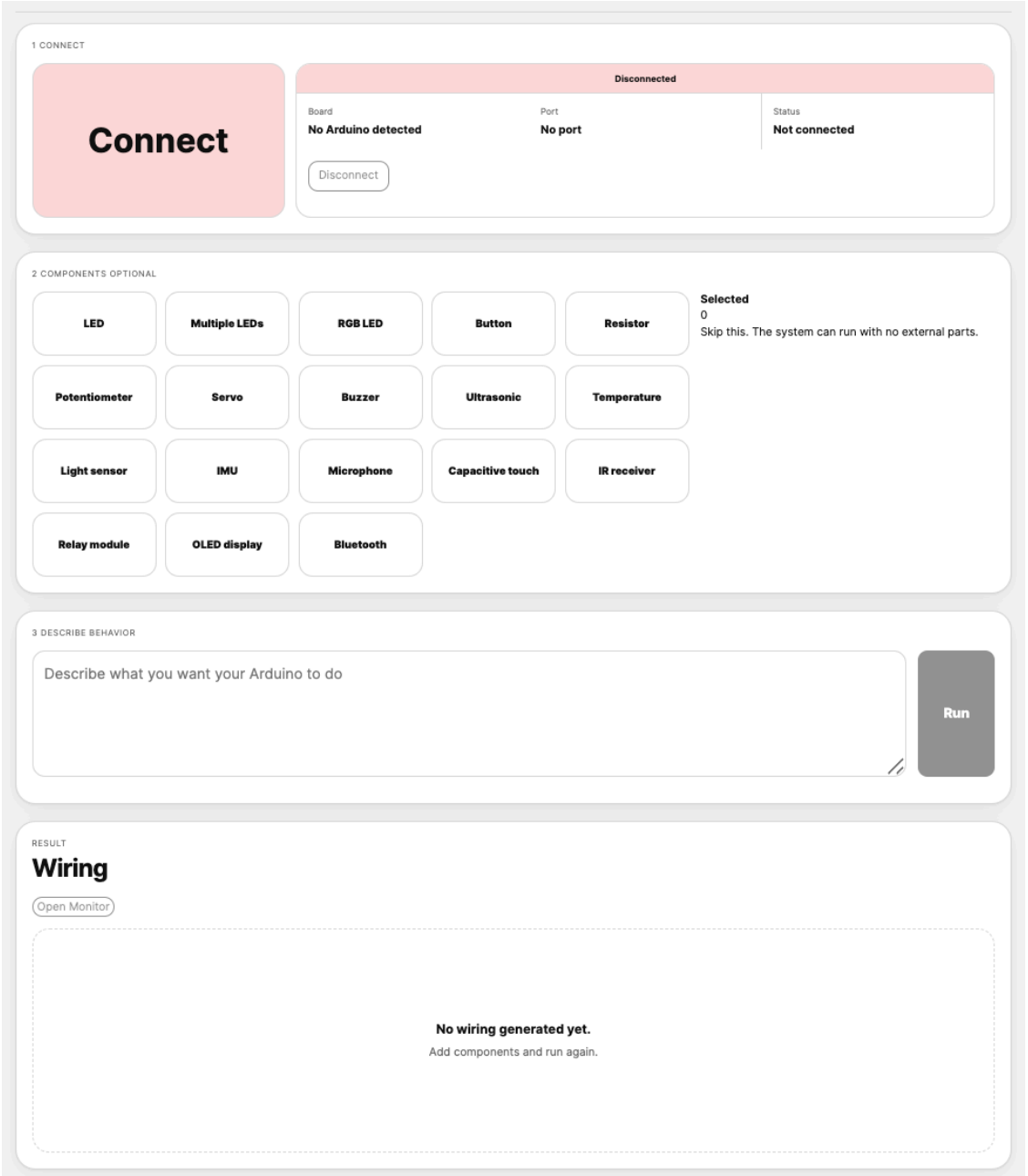


Figure 12: Overview of the website prototype interface showing the workflow for Arduino interaction, including device connection, component selection, natural language input, and wiring visualization.

Earlier stages of the research revealed that participants struggled less with generating code through AI and more with understanding wiring, hardware behavior, system feedback, and the relationship between abstract instructions and physical systems. All five participants also described experiences of information overload, uncertainty during interaction, and difficulty understanding technical terminology within existing workflows.

The design therefore focused on supporting transparency, step by step guidance, and embodied understanding in order to reduce uncertainty and make interaction outcomes easier to interpret for non technical users. Rather than requiring users to manage multiple technical decisions simultaneously, the interface was intentionally structured to guide interaction progressively through smaller sequential actions.

The interface was organised into multiple stages to reduce cognitive load, support iterative interaction, and help participants maintain orientation throughout the interaction process. This structure was directly informed by earlier testing sessions, where participants frequently relied on selective attention strategies and confirmation based interaction rather than attempting to understand the entire system simultaneously. The overall workflow of the prototype is shown in Figure 12.

Step 1 — Arduino Connection: The first stage focuses on establishing communication between the system and the Arduino board. Users connect the Arduino directly through the browser, after which the interface displays the connection status and device information as shown in Figure 13. This stage simplifies setup and reduces dependency on external development tools.



Figure 13: Arduino connection interface showing successful communication between the prototype and an Arduino Nano 33 BLE board using the Web Serial API.

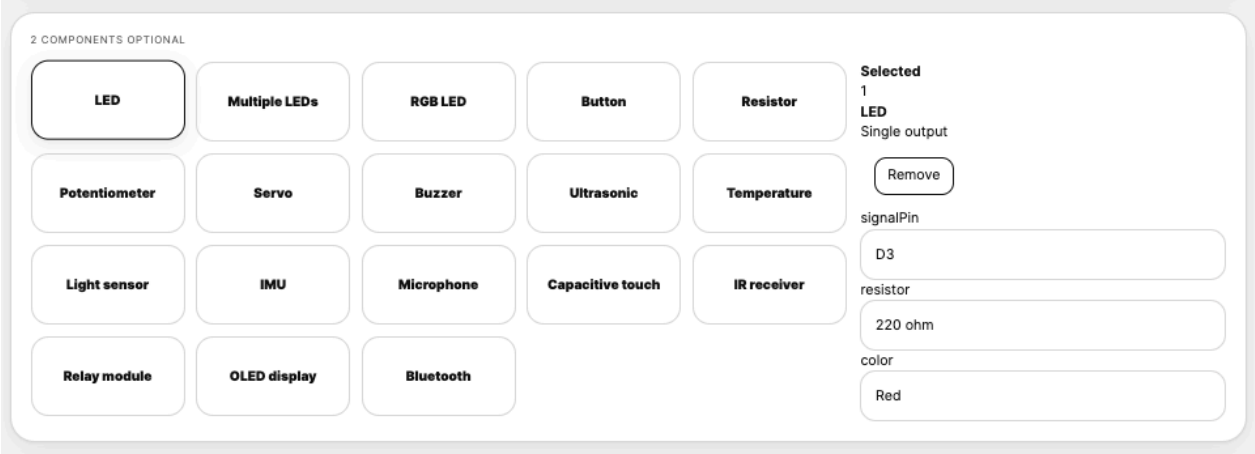


Figure 14: Component selection interface showing an LED module added and configured in the prototype workflow.

Step 2 — Component Selection: The second stage allows users to select electronic components such as LEDs, buzzers, buttons, sensors, and displays. Components are presented visually as selectable interface elements, allowing users to combine multiple hardware modules within the same project as shown in Figure 14. This stage was designed to make hardware configuration more accessible by shifting interaction away from technical specifications toward recognizable physical components.

The component selection stage was informed by interview and testing findings showing that participants struggled to interpret technical terminology and hardware configurations during interaction. Presenting components visually rather than

through technical specifications was intended to support recognition based interaction and reduce the cognitive load associated with physical computing concepts.

Step 3 — Prompt and Idea Generation: After selecting components, users describe intended behavior using natural language prompts. Instead of writing code directly, users specify goals conversationally, such as making an LED react to a button press or a buzzer respond to sensor input as shown in Figure 15. This interaction approach supports intent based interaction rather than manual programming.

Step 4 — AI Code Generation: Based on selected components and user prompts, the system automatically generates Arduino compatible code. This stage reduces the programming barrier by allowing users to focus on interaction goals and physical behavior rather than syntax and implementation details. The prototype therefore supports iterative prompting and experimentation rather than manual programming control.

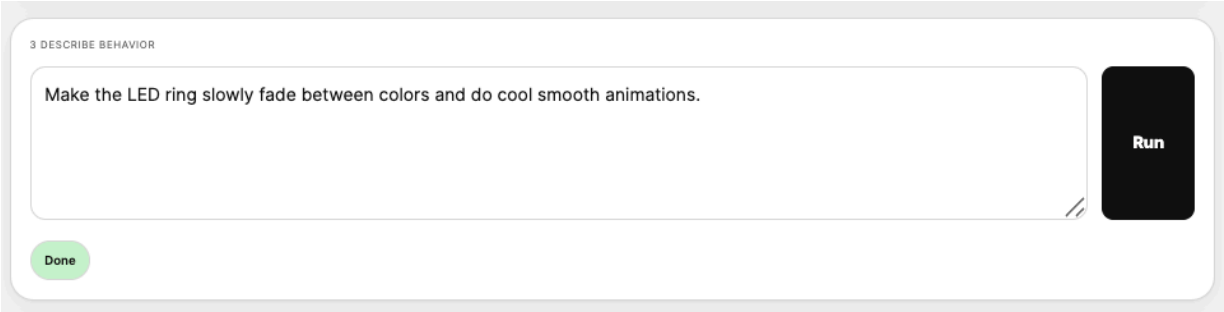


Figure 15: Natural language input interface where the user describes the intended Arduino behavior before AI assisted code generation.

Step 5 — Wiring Visualization: A central part of the prototype is the wiring visualization system. Earlier testing revealed that wiring and physical configuration represented the most significant challenge for participants. As a result, the prototype places strong emphasis on supporting physical understanding through visual guidance as shown in Figure 16.

The system generates a visual representation of how selected components should be connected to the Arduino board. Instead of static diagrams, the prototype dynamically overlays generated wiring paths onto images of a breadboard and Arduino Nano board. Different wire types are color coded to distinguish between power, ground, and signal connections, improving readability and reducing ambiguity during physical setup.

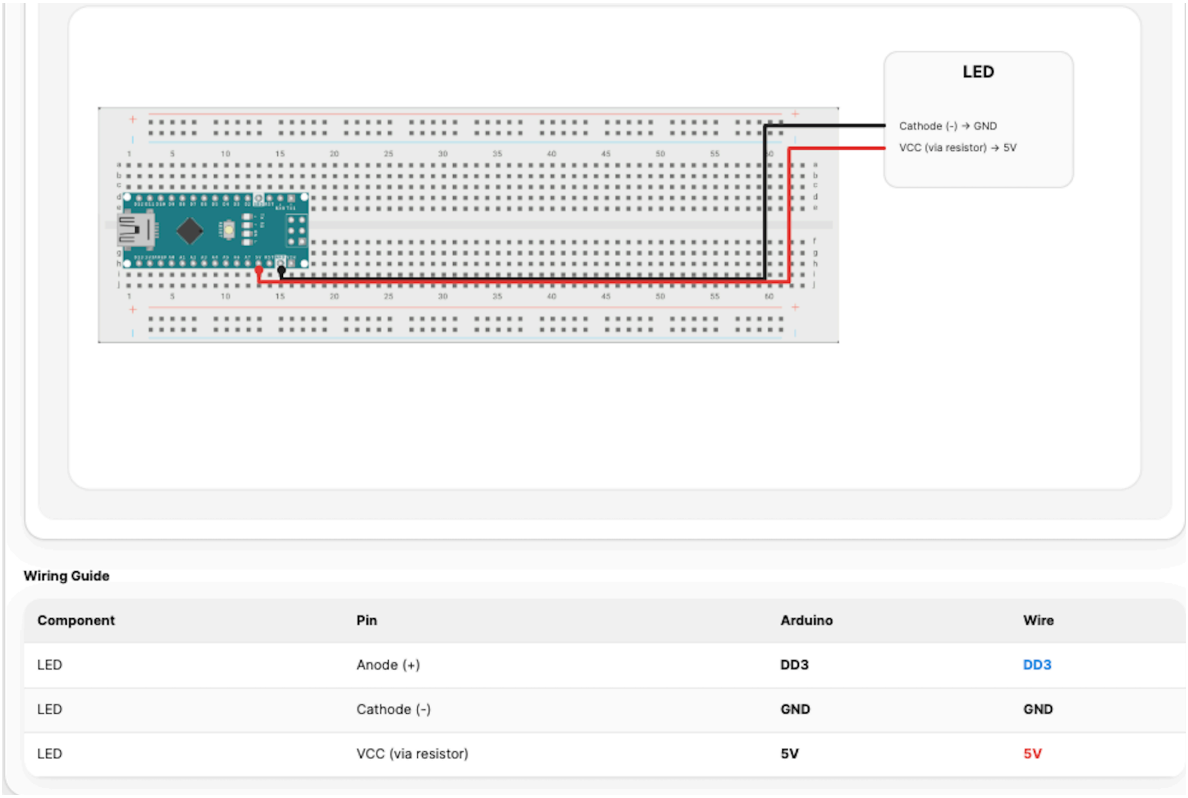


Figure 16: Wiring visualization showing component connections between the Arduino Nano board and an LED, including color coded signal, VCC, and GND wiring paths together with an automatically generated wiring guide listing component pins, Arduino pins,

In addition to the visual representation, the prototype also generates a structured wiring guide that lists the required component connections and corresponding Arduino pins. Combining visual diagrams with textual instructions was intended to support different forms of interpretation and reduce the gap between abstract instructions and embodied physical interaction.

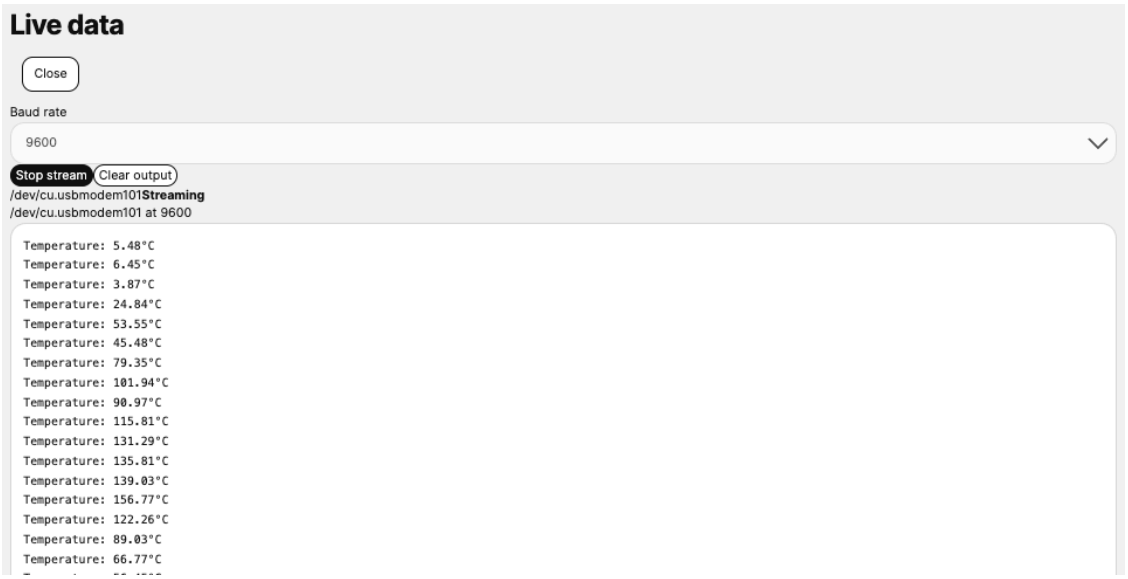


Figure 17: serial monitor displaying sensor output from the Arduino board directly within the prototype interface.

Step 6 — Serial Monitor and Interaction: The final stage includes an integrated serial monitor within the browser interface. Users can observe real time Arduino output, sensor values, and system feedback directly within the application as shown in Figure 17. This allows users to monitor system behavior, test interactions, and debug projects without switching between multiple tools. The inclusion of this stage was informed by the earlier finding that participants preferred maintaining interaction within a single continuous interface. This design decision specifically responded to participant frustration with switching between multiple tools and interfaces during earlier Codex CLI interaction and Arduino workflows.

From a technical perspective, the prototype was implemented using React.js together with the Web Serial API, enabling communication with Arduino boards connected through USB without requiring external development software. Connected devices were automatically identified through predefined board configurations, while the system architecture was organized into modular frontend and backend components handling functionality such as code generation, serial communication, component management, and wiring visualization.

The prototype also integrates the OpenAI API using the GPT-3.5 Turbo model to support AI assisted Arduino code generation through natural language prompts. Predefined system instructions were used to generate beginner friendly Arduino compatible code while reducing ambiguity and unsupported hardware behavior. The predefined model and system prompt configuration used for the AI generation workflow is shown below.

```
const model = process.env.OPENAI_MODEL || "gpt-3.5-turbo";

const systemPrompt = `
You are an expert Arduino programming assistant specialized in generating
complete Arduino compatible sketches for beginner level hardware projects.
Use the selected board type, connected components, and assigned pins to
generate accurate hardware interaction behavior. Only generate
functionality for the hardware components included in the provided project
configuration, and do not invent unsupported hardware behavior, pins, or
components.

Generate only executable Arduino C++ code without explanations, markdown
formatting, or code fences. Use LED_BUILTIN for onboard LED behavior
instead of hardcoded pin values such as pin 13. Ensure the generated code
is beginner friendly, readable, properly structured, and compatible with
Arduino IDE compilation. Return only the raw Arduino sketch.;
```

The wiring visualization system was implemented using dynamically generated SVG overlays combined with calibrated Arduino pin coordinates to align generated wiring paths with physical board representations. Overall, the prototype demonstrates how natural language interaction, AI assisted code generation, and physical computing can be combined into a unified interaction environment supporting beginner Arduino workflows.

6.6.1 Testing

Following the interviews and initial exploratory testing sessions, a later prototype evaluation stage was conducted with the same five participants (P1–P5). This stage evaluated how participants interacted with the developed prototype after earlier findings had been incorporated into the design.

Participants were asked to complete small Arduino tasks using the prototype while thinking aloud throughout the interaction process (see Appendix D). Participants were provided with an Arduino Nano 33 BLE starter kit containing a breadboard, wires, and electronic components to complete physical wiring tasks using the prototype's generated instructions and visualizations.

Four out of five participants described the prototype as easier to approach and less intimidating than previous Arduino development experiences. The sequential workflow structure helped participants understand where to begin and what to do next, reducing uncertainty during interaction. Four out of five participants stated that the prototype felt "*more friendly*" because interaction was visually separated into clear stages rather than presented through continuous streams of technical information and terminal feedback.

The Arduino connection stage was perceived positively. All five participants appreciated that the system automatically detected and connected to the Arduino board without requiring manual port selection or setup through external software. This reduced confusion during the initial interaction and simplified the setup process compared to previous experiences with the Arduino IDE. However, participants also encountered occasional inconsistencies within the wiring visualization system, where certain generated diagrams were missing individual wire connections, creating uncertainty during physical setup even though the overall interaction was still perceived as easier to follow.

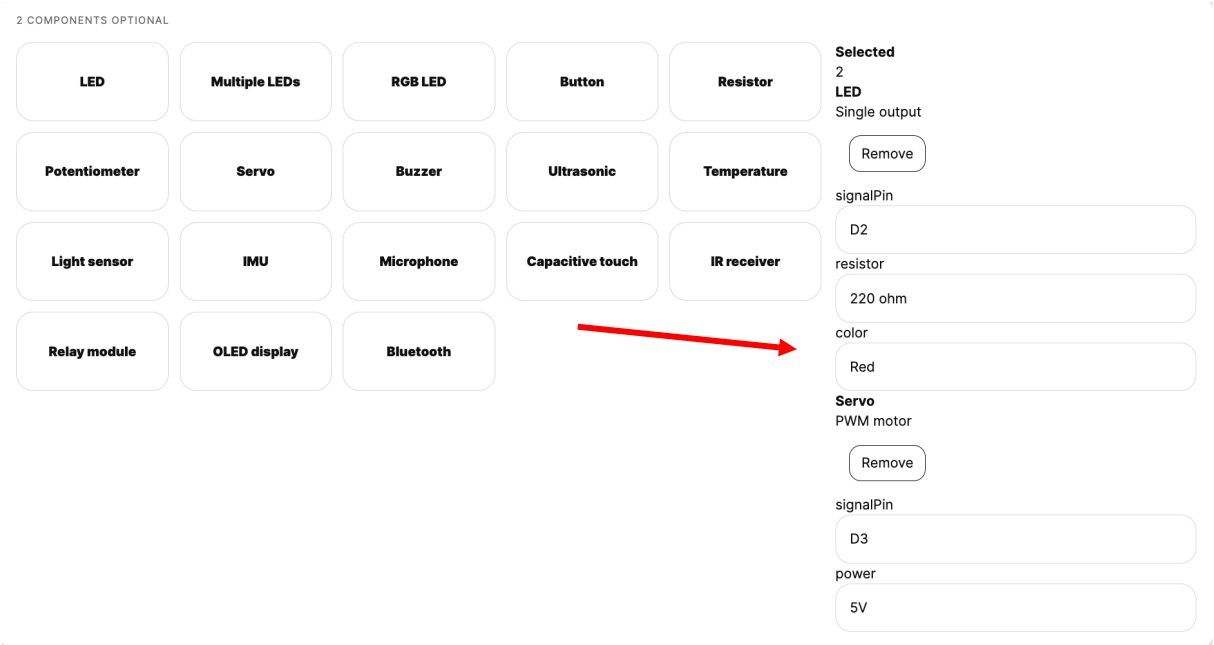


Figure 18: Component configuration interface showing signal pins, resistor values, and power settings that several participants found difficult to understand during interaction.

The component selection stage revealed both positive and negative reactions. All five participants appreciated being able to select recognizable hardware components visually. However, Three participants reported confusion regarding the configuration settings shown on the right side of the interface, particularly options such as signal pins, resistor values, and component parameters as seen in Figure 18. These settings

were often perceived as overly technical and difficult to understand for beginners. P2 stated, "*I don't understand this and I will probably never use it*" (P2) while P1 explained that they were unsure what values should be selected or why they mattered.

Three out of five participants also noted that the prototype contained many components they had never previously used or recognized. While two participants viewed this positively as something to explore later, three participants described the large selection as overwhelming. Three participants stated that they would prefer a smaller set of beginner focused components during initial interaction.

The natural language prompting stage was generally perceived as intuitive. All five participants were comfortable describing intended behavior conversationally and relied heavily on the AI generated code without attempting to fully understand the implementation. Similar to earlier findings, participants focused primarily on whether generated behavior functioned correctly rather than understanding the underlying implementation.

The wiring visualisation step represented the most positively received aspect of the prototype. All five participants consistently stated that the visual wiring diagrams were easier to understand than previous text based instructions or external diagrams found online. The combination of color coded wires and the generated wiring guide helped participants connect abstract instructions to physical hardware placement. Two participants (P1 and P4) described the wiring visualization as reducing uncertainty during physical setup and making the interaction feel "*less scary*" compared to earlier Arduino experiences. Despite this improvement, some participants still struggled to understand the meaning of concepts such as signal pins, power connections, and resistor placement. This suggests that while the visualization improved accessibility, underlying hardware concepts remained difficult for beginners to fully interpret independently.

The serial monitor stage produced mixed reactions. Participants appreciated being able to view live sensor values and feedback directly within the browser interface without opening the Arduino IDE. However, Three participants commented that the serial monitor looked different from what they were accustomed to inside the standard Arduino IDE serial monitor interface. This reflects the influence of existing Arduino platform conventions and interaction patterns discussed earlier in Section 2.7.3, where participants' expectations were shaped by prior experience with traditional Arduino development environments.

Overall, the testing suggested that the prototype reduced barriers related to interface complexity, wiring uncertainty, and switching between multiple tools. At the same time, the findings also revealed that beginners still struggle with understanding hardware terminology, electrical concepts, and component configuration details even when interaction is visually guided and AI assisted. This testing phase therefore reinforced one of the central findings of the thesis: the primary challenge in AI assisted physical computing is not necessarily generating code, but supporting users in understanding and translating abstract system behavior into embodied physical interaction. Similar to systems such as ViperGPT, the prototype explored how AI agents can mediate between natural language input and executable system behavior, translating conversational prompts into actionable operations within physical computing environments.

While participants were generally comfortable relying on AI generated code and natural language interaction, they continued to experience uncertainty when interpreting hardware configurations, electrical concepts, and physical system behavior. The findings suggest that reducing programming complexity alone is insufficient if users still struggle to understand how digital instructions relate to real world physical interaction and material behavior.

6.6.2 Iteration

Following the prototype evaluation stage described in Section 6.6.1, the prototype was iteratively refined based on the interaction difficulties, participant feedback, and usability issues identified during testing. The changes focused primarily on improving clarity, reducing beginner confusion, and strengthening the relationship between generated instructions and physical interaction. The iteration phase concentrated on simplifying interaction and improving the reliability of the existing workflow.

The component selection stage underwent significant redesign following participant feedback from the evaluation stage in Section 6.6.1. During testing, four out of five participants experienced confusion regarding the configuration panel shown beside selected components, particularly options related to signal pins, resistor values, and hardware parameters. Participants frequently described these settings as overly technical and unnecessary for beginner interaction. In response, the configuration interface was removed entirely and replaced with a simplified interaction flow focused only on selecting physical components visually. This redesign reflected a broader interaction design decision to prioritise physical exploration and experimentation over technical configuration. Earlier testing suggested that exposing beginners to low level hardware parameters too early shifted attention away from interaction goals and increased uncertainty during setup.

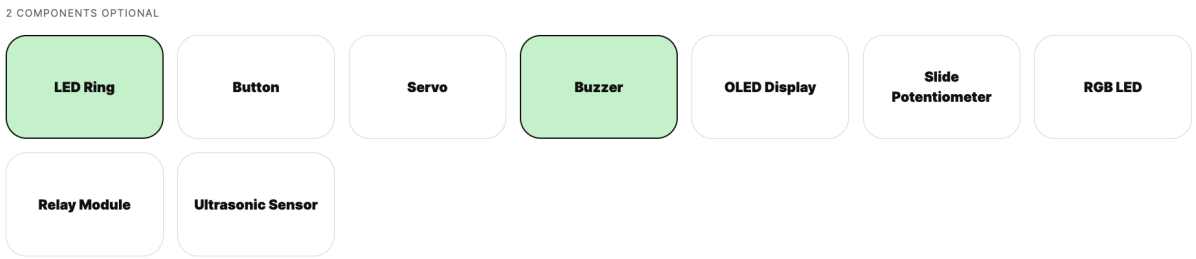


Figure 19: Redesigned component selection interface showing the simplified beginner focused starter kit and updated visual selection system introduced after testing.

The component step was redesigned to simplify interaction for beginners. Earlier testing revealed that participants found the large number of unfamiliar components difficult to navigate during initial interaction. As a result, the prototype was restructured around a smaller beginner focused starter kit containing only commonly used and recognizable components such as LED rings, buttons, buzzers, servo motors, OLED displays, ultrasonic sensors, and RGB LEDs. Selected components were visually highlighted using the same color already used within the Arduino connection stage, improving visual consistency across the interface. Users could also toggle components directly through the component tiles to simplify experimentation. Figure 19 shows the redesigned component selection interface.

One of the most significant issues identified during testing concerned the wiring visualization system. Although participants described the visual wiring diagrams as the most useful part of the prototype, several inconsistencies were discovered during physical testing. In some cases, generated wiring diagrams contained incorrect or missing wire connections, particularly when multiple components were connected simultaneously. This created uncertainty during the physical setup process.

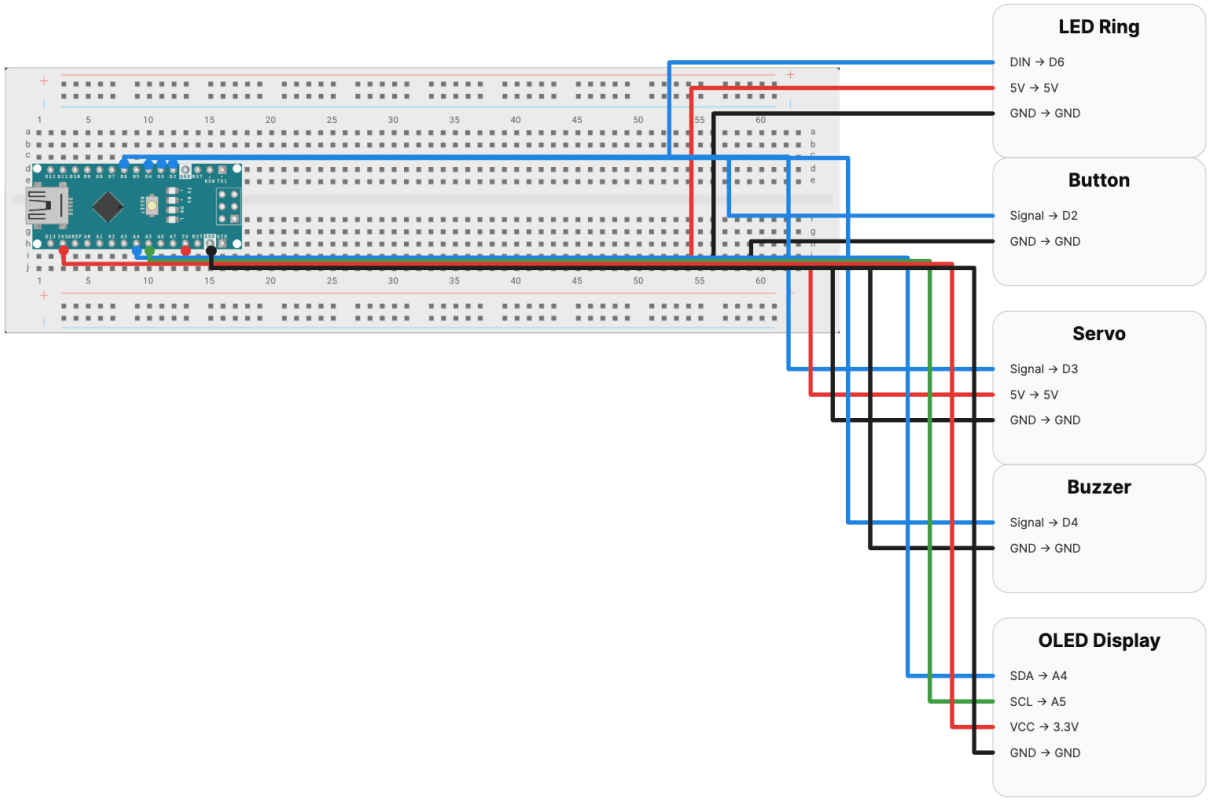


Figure 20: Updated wiring step showing improved wire routing, corrected Arduino pin alignment, expanded multi component support, and clearer separation between signal, power, and ground connections.

To address this, The wiring visualization system was recalibrated to improve alignment between generated wires and the Arduino Nano board representation. Wire paths were redesigned to reduce overlap and improve readability during multi component setups. Additional spacing was introduced within the visualization layout to better support more complex projects involving multiple components. The generated wiring system was also extended to support additional Arduino pins, analog connections, OLED communication lines, and more advanced component combinations. Figure 20 shows the updated wiring visualization system after iteration.

The serial monitor interface was also redesigned following participant reactions during testing. Although participants appreciated being able to monitor live Arduino output directly within the browser, several commented that the interface looked unfamiliar compared to the standard Arduino IDE serial monitor. In response, the serial monitor was redesigned into a more structured terminal style interface intended to feel clearer and more recognizable while preserving the workflow.

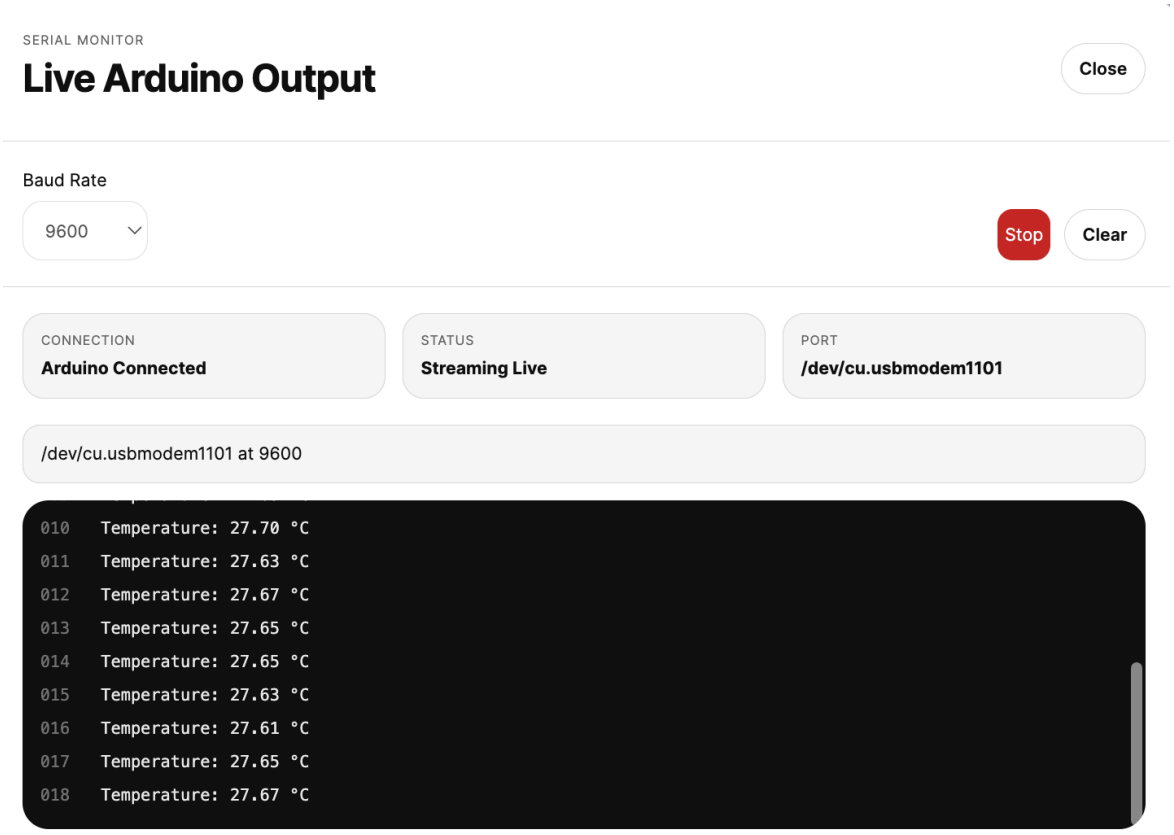


Figure 21: Redesigned serial monitor interface.

The updated serial monitor introduced clearer connection feedback, improved terminal readability, automatic scrolling during live output, simplified baud rate controls, and real time connection status indicators. The system was also refined to better manage live data streaming and monitor cleanup during repeated interactions.

In addition, the prototype was extended to support real time sensing using the built in temperature and humidity sensor available on the Arduino Nano 33 BLE Sense board. This allowed participants to observe live sensor feedback directly within the browser without additional hardware components. Figure 21 shows the redesigned serial monitor interface. The iteration phase also included improvements to the AI generation workflow itself. Earlier testing demonstrated that participants were highly outcome driven and relied heavily on AI generated behavior without understanding the underlying implementation.

To improve reliability during common beginner tasks, local generation logic was introduced for built in sensing interactions such as temperature and humidity monitoring. Instead of relying entirely on GPT generated code, the system detects environmental prompts and generates predefined Arduino sketches for the onboard Nano 33 BLE Sense sensors. This reduced inconsistent AI output and improved the stability of beginner interactions involving sensor data streaming. Overall, the iteration phase focused on reducing interaction friction while simplifying interaction, improving physical clarity, and strengthening the relationship between natural language input and generated system behavior.

7 Design Outcomes Summary

This section summarizes the main design outcomes that emerged through the research through design process. The prototype was iteratively shaped through literature research, technical exploration, interviews, testing, thematic analysis, and refinement. Throughout the project, the prototype functioned both as a design artifact and as a way of investigating how users interact with AI agents within physical computing contexts.

A central design outcome was the shift from traditional code focused interaction toward intent based interaction. Earlier findings showed that participants already relied heavily on AI tools to generate Arduino code. The prototype therefore focused less on replacing manual coding and more on integrating prompting, code generation, wiring guidance, and physical interaction into a single workflow. Instead of separating programming, hardware setup, debugging, and testing across multiple tools, the final system combined these activities within one continuous interaction environment.

Another important outcome was the emphasis on visual guidance throughout the interaction process. Earlier testing showed that participants struggled more with wiring and physical understanding than with generating code. The prototype therefore prioritised wiring visualization, system feedback, and sequential interaction flows to support interpretation and physical understanding. These design decisions aimed to reduce uncertainty during interaction and help users better connect abstract instructions to observable physical behavior.

The project also revealed that interaction with AI agents in physical computing becomes highly iterative, where users continuously refine prompts, test generated behavior, and adjust physical configurations based on system feedback. Rather than interacting through fixed programming procedures, participants relied on experimentation, conversational prompting, and repeated refinement throughout the interaction process.

Overall, the project demonstrated how AI agents, natural language interaction, and physical computing can be combined into a unified interaction system supporting beginner Arduino workflows. At the same time, the findings highlight that future AI assisted physical computing systems must support not only code generation, but also interpretation, transparency, and user understanding of system behavior.

8 Discussion/Evaluation

This section discusses and evaluates the findings and final outcomes of the project in relation to the research questions, theoretical framework, and research through design process. The discussion reflects on how participants interacted with AI agents in physical computing contexts, how the prototype supported AI agents and physical computing and natural language interaction based workflows, and what challenges remained throughout the design process. The section also critically evaluates the final prototype, reflects on the iterative design process, and discusses implications for future AI driven physical computing systems.

8.1 AI Agents and Embodied Interaction

This thesis explored how AI agents and natural language interaction influence beginner interaction with physical computing systems. Through literature research, interviews, prototype development, testing, and iterative refinement, the project investigated how users shape, interpret, and negotiate physical system behavior when interaction shifts from manual coding toward AI mediated workflows.

One of the central findings of the project is that AI agents reduce barriers related to programming syntax and code production, but do not remove challenges related to embodied interaction and physical understanding. All five participants consistently expressed greater confidence when relying on AI generated code than when interacting with wiring, electrical concepts, or physical hardware configuration. This suggests that the primary difficulty in beginner oriented physical computing is no longer necessarily writing code itself, but understanding how abstract digital instructions translate into physical system behavior.

This finding connects closely to theories of embodied interaction discussed in the background chapter. Dourish (2001) argues that understanding emerges through engagement with the physical world rather than through abstract computational processes alone. The findings support this perspective. Four out of five participants rarely evaluated generated code directly and instead interpreted success or failure primarily through observable physical outcomes or wiring configurations functioning correctly.

The findings also align with recent human AI interaction research highlighting that successful AI assisted systems depend not only on automation capabilities but also on intelligibility, transparency, and the user's ability to meaningfully interpret AI generated outcomes (Ehsan et al., 2021; Shneiderman, 2022).

8.2 Iterative Interaction, Trust, and Control

The project revealed that interaction with AI agents in physical computing becomes highly iterative and conversational. Rather than constructing behavior manually through code, all five participants continuously refined prompts, tested generated outputs, interpreted system behavior, and adjusted interaction based on feedback from both the AI system and the physical hardware. This reflects a broader shift from code focused interaction toward intent based interaction.

At the same time, AI generated interaction introduced new forms of uncertainty. While participants generally trusted AI generated programming logic, confidence decreased when interaction involved ambiguous wiring instructions, unexpected hardware behavior, or inconsistencies between generated outputs and physical results. In these situations, participants relied on experimentation, visual interpretation, and repeated testing to regain understanding and control. This suggests that trust in AI assisted physical computing is negotiated dynamically through interaction rather than established automatically through system authority alone. This reflects earlier discussions on collaborative human AI interaction, where transparency and interpretability influence how users negotiate trust and control during interaction (Shneiderman, 2022; Ehsan et al., 2021).

8.3 Automation, Transparency, and Learning

The findings revealed a tension between automation and transparency. AI agents simplified programming tasks, but increased abstraction between user intention and system execution. All five participants were often able to generate functioning behaviors without fully understanding how the generated code or hardware interactions operated internally. While this supported accessibility and rapid experimentation, it also created difficulties when diagnosing failures or interpreting unexpected behavior.

Another important finding concerns the role of visual guidance within AI assisted physical computing systems. Participants consistently described the wiring visualization step as easier to interpret than terminal output or text based instructions alone. This suggests that future AI assisted physical computing systems may benefit from focusing not only on intelligent code generation, but also on supporting spatial reasoning, material understanding, and embodied interpretation through visual and interactive feedback. The findings also raise broader questions regarding learning and understanding within AI assisted physical computing. Although the prototype reduced barriers related to programming syntax and code generation, participants still needed to understand several fundamental physical computing concepts in order to use the system successfully. These included understanding how breadboards function, how components are physically connected, the role of VCC and GND connections, signal flow, and how hardware behavior changes based on wiring configuration.

At the same time, the role of learning changed significantly within the interaction process. Rather than learning programming through manually writing code line by line, participants primarily interacted through prompting, experimentation, observation, and refinement. The system therefore shifted interaction away from traditional syntax based programming toward higher level intention based interaction. This aligns with the earlier discussion on intent based interaction and end user programming (Lieberman et al., 2006; Shneiderman, 2022).

This created an important distinction between task completion and deeper technical understanding. Participants were often able to complete Arduino tasks successfully through AI generation and visual guidance without fully understanding the generated code, electrical principles, or hardware logic involved in the interaction. In this sense, the prototype supported accessibility and rapid experimentation, but did not necessarily ensure conceptual understanding of electronics or programming fundamentals.

Compared to more traditional approaches to electronics education, where users manually construct circuits, troubleshoot wiring, and write programs step by step, the interaction model explored in this project changes what users need to learn. Instead of focusing primarily on syntax memorization or low level implementation details, interaction increasingly depends on interpreting physical feedback, understanding component relationships, and evaluating whether generated behavior matches user intention.

8.4 Evaluation of the Prototype and Design Process

The prototype itself should also be critically evaluated. While the system successfully integrated prompting, code generation, wiring visualization, and hardware interaction into a unified interaction workflow, several limitations remained throughout development. Generated outputs occasionally produced inconsistencies in code suggestions and wiring instructions, wiring visualizations sometimes required recalibration, and participants still experienced uncertainty regarding hardware terminology and electrical concepts even after iterative refinement. These findings suggest that improving automated generation alone is insufficient for supporting beginner interaction in physical computing contexts.

From a research through resign perspective, the project demonstrates how prototype development itself functioned as a form of inquiry. The iterative relationship between testing, reflection, and redesign repeatedly exposed new interaction challenges and directly informed later design decisions. Participant feedback continuously redirected the project away from purely technical optimization and toward issues related to interpretation, uncertainty, and embodied understanding. This reflects the iterative and reflective principles of research through design discussed earlier in the methodological framework (Zimmerman et al., 2007). At the same time, the design process also revealed limitations. Because the prototype evolved continuously throughout development, several interaction issues were only identified during later testing stages after substantial technical implementation had already been completed. The project also demonstrated the difficulty of balancing technical flexibility with beginner oriented simplicity when integrating AI generated behavior with physical hardware interaction.

Another limitation concerns the participant group used throughout the study. The same five participants were intentionally involved across the interviews, testing sessions, and later prototype evaluations in order to support continuity throughout the iterative research through design process. While this allowed interaction patterns and behavioral changes to be observed over time, repeated exposure to the system may also have influenced participant familiarity with the system, confidence, and expectations during later testing stages.

Another important consideration concerns the role of AI assisted development tools within the design process. While the prototype design was primarily informed by interviews, testing, thematic analysis, and iterative refinement, tools such as Codex CLI and GPT models supported parts of the technical implementation and exploration process. This reflects how AI assisted tools can influence development workflows and technical experimentation within interaction design practice without replacing the role of the designer in shaping the overall interaction direction and design decisions.

8.5 Research Questions Reflection

The findings of the project provide several insights into how users interact with AI agents in physical computing contexts.

RQ1 explored how users iteratively shape the behavior of a physical computing system through natural language interaction with an AI agent. The findings showed that users shaped behavior through repeated cycles of prompting, testing, observing outcomes, and refining interaction based on feedback from both the AI system and the physical hardware.

RQ2 examined how users interpret and respond to mismatches between intended and AI generated behavior. The findings demonstrated that participants primarily interpreted mismatches through observable physical outcomes such as incorrect wiring, failed hardware responses, or unexpected component behavior. Users relied heavily on experimentation, visual feedback, and iterative testing rather than analyzing generated code directly.

RQ3 investigated how users negotiate control with AI agents during physical computing interaction. The findings suggest that control emerged through continuous negotiation between user intention, AI generated outputs, and physical system behavior. While participants generally trusted AI generated programming logic, they became more critical and exploratory when interacting with physical hardware, wiring, and unexpected system responses.

9 Future work

While the prototype demonstrates how AI agents and natural language interaction can support beginner oriented physical computing workflows, several areas remain open for future development and refinement.

One important direction concerns expanding hardware compatibility. The current prototype primarily supports the Arduino Nano 33 BLE Sense and Nano Rev2 boards. Future versions could support a broader range of Arduino boards and microcontrollers while automatically adapting the interface, available components, and wiring logic based on the connected hardware. This could create a more dynamic and context aware interaction workflow where the system responds directly to the capabilities of the detected board.

Another important area involves improving the reliability and flexibility of the wiring visualization system. Although the current implementation successfully supports beginner interaction through generated wiring overlays, future iterations could move toward fully dynamic SVG based visualizations with more accurate pin alignment, improved wire routing, automatic scaling, and support for more complex multi component configurations. Ensuring that generated wiring remains visually accurate and technically reliable is critical for supporting trust and independent use.

Future versions could also expand the relationship between AI generation and user understanding. The current prototype focuses primarily on reducing interaction barriers through automation and visual guidance. Later iterations could additionally allow users to inspect, modify, and learn from the generated Arduino code directly within the interface. This could support both accessibility and gradual technical learning by allowing users to move between high level prompting and lower level implementation details when desired.

The current system also treats prompts primarily as isolated generation requests. Future versions could instead support continuous conversational refinement, where users iteratively modify existing Arduino projects through ongoing dialogue with the AI agent rather than generating entirely new sketches after each interaction. This could strengthen the collaborative and exploratory nature of AI assisted physical computing workflows.

Finally, longer term evaluations involving participants with different technical backgrounds could provide deeper understanding of how trust, learning, control, and embodied interaction evolve over time when interacting with AI assisted physical computing systems. Since the current project primarily involved Interaction Design students, future research could also explore how similar systems might support learners in primary education, secondary education, computer science education, or non technical university disciplines. Such contexts would likely require different interaction models, levels of guidance, and pedagogical support. Future work should therefore also involve didactic and pedagogical perspectives to better evaluate how AI assisted physical computing systems support learning, understanding, and long term knowledge development rather than only task completion and accessibility.

10 Conclusion

This thesis explored how AI agents and natural language interaction can support beginner interaction with physical computing systems. Using a research through design approach, the project combined literature research, interviews, prototype development, testing, thematic analysis, and iterative refinement to investigate how users shape, interpret, and negotiate physical system behavior through AI assisted workflows.

The findings demonstrated that AI agents reduce barriers related to programming and code production, but do not remove challenges related to embodied interaction and physical understanding. Participants were generally comfortable relying on AI generated code, yet continued to experience uncertainty when interpreting wiring, hardware behavior, and physical system responses. As a result, the project increasingly shifted toward supporting interpretation, visual guidance, and real time feedback rather than programming alone.

The project also revealed that interaction with AI agents in physical computing becomes highly iterative. Participants continuously refined prompts, tested generated behavior, and adjusted interaction based on physical system feedback. Trust in AI generated behavior was shown to depend strongly on context, particularly when interaction involved physical hardware and wiring.

The final prototype demonstrates how AI agents, natural language interaction, and physical computing can be combined into a unified interactive website interaction system for beginner oriented embodied interaction. The project contributes interaction design knowledge for understanding how users collaborate with AI agents when shaping physical system behavior through natural language.

Nevertheless, the findings suggest that AI assisted physical computing systems represent a broader shift in how users interact with computational systems through natural language and physical interaction. Rather than directly programming behavior through code alone, users increasingly operate through iterative cycles of prompting, interpretation, testing, and refinement in collaboration with AI agents. The project therefore contributes design knowledge for this emerging interaction space, highlighting the importance of transparency, feedback, and physical understanding when AI generated behavior is expressed through physical systems.

11. References

Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S. T., Bennett, P. N., Inkpen, K., Teevan, J., Kikin-Gil, R., & Horvitz, E. (2019). *Guidelines for human-AI interaction. Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, 1–13. https://doi.org/10.1145/3290605.3300233

Banzi, M., & Shiloh, M. (2014). *Getting started with Arduino* (3rd ed.). Maker Media.

Booth, A., Sutton, A., & Papaioannou, D. (2016). *Systematic approaches to a successful literature review* (2nd ed.). Sage.

Braun, V., & Clarke, V. (2006). Using thematic analysis in psychology. *Qualitative Research in Psychology, 3*(2), 77–101. https://doi.org/10.1191/1478088706qp063oa

Dellermann, D., Ebel, P., Söllner, M., & Leimeister, J. M. (2019). Hybrid intelligence. *Business & Information Systems Engineering, 61*(5), 637–643. https://doi.org/10.1007/s12599-019-00595-2

Dourish, P. (2001). *Where the action is: The foundations of embodied interaction.* MIT Press.

Ehsan, U., Liao, Q. V., Muller, M., Riedl, M. O., & Weisz, J. D. (2021). Expanding explainability: Towards social transparency in AI systems. *CHI Conference on Human Factors in Computing Systems*, 1–19. https://doi.org/10.1145/3411764.3445188

Ericsson, K. A., & Simon, H. A. (1993). *Protocol analysis: Verbal reports as data* (Rev. ed.). MIT Press.

European Union. (2016). *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation).* https://eur-lex.europa.eu/eli/reg/2016/679/oj

Houde, S., & Hill, C. (1997). What do prototypes prototype? In M. G. Helander, T. K. Landauer, & P. V. Prabhu (Eds.), *Handbook of human-computer interaction* (2nd ed., pp. 367–381). Elsevier.

Ishii, H., & Ullmer, B. (1997). Tangible bits: Towards seamless interfaces between people, bits, and atoms. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 234–241. https://doi.org/10.1145/258549.258715

Kvale, S., & Brinkmann, S. (2009). *Interviews: Learning the craft of qualitative research interviewing* (2nd ed.). Sage.

Lieberman, H., Paternò, F., & Wulf, V. (Eds.). (2006). *End-user development*. Springer.

Löwgren, J. (2007). Interaction design, research practices and design research on digital materials. In S. Ilstedt Hjelm (Ed.), *Under ytan: Om designforskning* (pp. 85–106). Raster Förlag.

Myers, B. A., Ko, A. J., & Burnett, M. M. (2006). Invited research overview: End-user programming. *CHI Extended Abstracts on Human Factors in Computing Systems*, 75–80. https://doi.org/10.1145/1125451.1125456

O'Sullivan, D., & Igoe, T. (2004). *Physical computing: Sensing and controlling the physical world with computers*. Course Technology Press.

Shneiderman, B. (2022). *Human-centered AI*. Oxford University Press.

Surís, D., Menon, S., Vondrick, C., & Russell, B. (2023). ViperGPT: Visual inference via Python execution for reasoning. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 11888–11898. https://doi.org/10.1109/ICCV51070.2023.01093

Swedish Research Council. (2017). *Good research practice*. https://www.vr.se/english/analysis/reports/our-reports/2017-08-31-good-research-practice.html

Vaithilingam, P., Zhang, T., Glassman, E. L., Sarkar, A., & Kim, J. (2022). Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. *CHI Conference on Human Factors in Computing Systems*. https://doi.org/10.1145/3491102.3501819

Wenger, M. J., & Spyridakis, J. H. (1989). The relevance of protocol analysis to usability testing. *Behaviour & Information Technology, 8*(3), 223–233. https://doi.org/10.1080/01449298908914533

Zimmerman, J., Stolterman, E., & Forlizzi, J. (2007). Research through design as a method for interaction design research. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 493–502. https://doi.org/10.1145/1240624.1240704

12. Appendix

Appendix A — Consent Form

Consent Form

Researcher: Anas Aljoudi
Malmö University
Interaction Design MSc Programme
Thesis Project

I agree to participate in this thesis study, which explores how beginners interact with Arduino systems and AI assisted tools such as CodeX through a command-line interface (CLI). The purpose of this research is to understand how users approach programming, wiring, and electronic components, and to identify challenges in learning and using these systems.

I understand that my participation will involve an interview and a short hands on session where I may be asked to interact with Arduino related tasks and a CLI based tool. During this session, the researcher may observe my interactions and take notes.

I understand that I will be asked about my experience with Arduino, including programming, wiring, and working with components, as well as my experience using the Codex CLI tool. This may include questions about difficulties, expectations, and overall understanding.

I understand that my responses will be used to identify patterns in how beginners understand and work with Arduino concepts and AI assisted tools.

I understand that all collected data will be treated confidentially and anonymized. My identity will not be disclosed, and the data will only be used for the purposes of this thesis project.

I understand that my participation is voluntary, and I may withdraw at any time without any consequences. I may also choose not to answer any question or stop the session at any point.

Name:

Date:

Signature:

Appendix B — Interview Questions

Q1: *Self assessment of experience "How would you rate your experience with Arduino programming and wiring on a scale from 1 to 5, where 1 means no experience and 5 means advanced? Please explain why you chose that number."*

Q2: *Previous experience and projects "What have you done so far with Arduino? Can you describe any projects or prototypes you have worked on, and what your role was in building them?"*

Q3: *Mental model of Arduino systems "Can you describe what happens when you upload code to an Arduino and connect it to a circuit?"*

Q4: *Perceived difficulty areas "When working on an Arduino project, which parts do you find most confusing or difficult? For example, do you find the hardware side (wiring, components) or the software side (programming) more challenging? Please explain why."*

Q5: *Understanding of wiring and components "If you were asked to connect an LED to an Arduino, how would you approach it, and what do you think each component in the circuit does?"*

Q6: *Expectations of Codex CLI "When using a tool like Codex in a command-line interface, what would you expect it to help you with, and what feels unclear or challenging about using it?"*

Q7: *Perceived readiness for AI assisted tools "What do you think someone needs to understand before they can effectively use a tool like Codex for Arduino projects?"*

Appendix C — Initial Testing Tasks

Task 1 — LED Blinking

Prompt Codex CLI to generate code that makes the onboard LED blink continuously.

Task 2 — Temperature Sensor Feedback

Use the built-in temperature sensor to print live temperature values to the serial monitor.

Task 3 — Interactive LED Behavior

Generate code that changes the LED blinking speed based on temperature sensor readings.

Task 4 — Motion Detection

Use the onboard IMU sensor to detect movement and trigger an LED response.

Task 5 — Sound or Microphone Interaction

Generate code that reacts to microphone input through changes in LED behavior.

Appendix D — Final Prototype Testing Tasks

Task 1 — Connect and Configure an LED

Use the prototype to add an LED component, configure the required settings, and generate the corresponding wiring visualization and Arduino behavior.

Task 2 — Generate Interactive LED Behavior

Use natural language prompting to create an LED interaction based on user defined behavior such as blinking or brightness control.

Task 3 — Connect and Use a Sensor

Add a sensor component and generate code that responds to live sensor data through physical output or serial monitor feedback.

Task 4 — Interpret Wiring Visualization

Follow the generated wiring diagram and physically connect the components using the breadboard and Arduino board.

Task 5 — Upload and Test Generated Behavior

Upload the generated code to the Arduino board and evaluate whether the physical behavior matches the intended interaction.

Appendix E — Prototype GitHub Repository

GitHub link: https://github.com/Anasaljoudi/AI-Agents-in-Physical-Computing

Appendix F — Video Prototype

YouTube Link: https://www.youtube.com/watch?v=FDccRFuWnQE